

React js

Many popular websites and web applications are built with React due to its flexibility, performance, and scalability. Here are some well-known examples:

1. Facebook

- React was originally developed by Facebook, and it's used extensively across the platform. Both the Facebook website and the Instagram app use React in their user interfaces.

2. Instagram

- Instagram's web app is built using React, including features like the Explore page, photo/video uploads, and direct messaging.

3. Netflix

- Netflix uses React on its client-side for fast and smooth performance. The front-end architecture of their platform makes use of React for its user interface rendering.

4. Airbnb

- Airbnb uses React for its rich, interactive user experience. It powers many of their features, from search and filters to booking management.

5. WhatsApp Web

- The web version of WhatsApp, which offers messaging services directly from a browser, is built using React.

6. Uber

- Uber's web dashboard for managing rides and the admin panel for business management are built using React, providing a responsive and fluid experience.

7. Dropbox

- Dropbox uses React to manage its complex user interface and provide a seamless file-sharing experience.

8. BBC

- The British Broadcasting Corporation (BBC) uses React on its website to deliver content-rich pages with interactive elements and smooth navigation.

9. New York Times

- The New York Times uses React for interactive articles, multimedia content, and its crossword puzzle feature.

10. Atlassian Products (Jira, Confluence, Bitbucket)

- Atlassian's suite of products, including Jira, Confluence, and Bitbucket, use React to deliver rich, interactive user experiences for project management and collaboration.

11. GitHub

- GitHub's new UI, especially the code editor and notifications page, makes use of React to enhance performance and usability.

12. Reddit

- Reddit's redesign was done using React. The modern, responsive interface you see today is powered by React on both desktop and mobile versions.

13. Shopify

- Shopify uses React for its admin interface, providing a dynamic and interactive experience for shop owners managing their stores.

14. Trello

- Trello uses React for its web app, offering a smooth drag-and-drop interface for task and project management.

15. CodeSandbox

- CodeSandbox, an online code editor for web applications, is built using React. It allows developers to write, test, and share React code directly in the browser.

16. Discord

- Discord's web app uses React for rendering its user interface, allowing for real-time updates and seamless user interactions.

17. Salesforce

- Salesforce's Lightning Design System, used for building enterprise applications, is built with React.

18. Khan Academy

- Khan Academy uses React to create interactive and educational content, enabling a dynamic and responsive learning experience.

19. Pinterest

- Pinterest's web interface is built using React, ensuring fast and smooth interactions, especially when browsing through a large number of images.

20. Microsoft Products (Outlook, Teams)

- Microsoft has used React for its web versions of Outlook and Teams, providing a responsive and interactive user experience.

Prerequisites for Learning ReactJS

1. HTML and CSS

HTML and CSS are the pillars for all front-end developers. So, before you start learning ReactJS, make sure you know how to code in HTML and CSS.

You should be able to write semantic HTML tags, and CSS selectors, use classes, enforce a CSS reset, box model, reset to border-box, flexbox, write responsive web applications with media queries, and create a frontend framework using HTML and CSS.

2. Node Package Manager

The Node Package Manager plays a role in the installation and management of JavaScript applications. Installing NPM is as easy as installing Node.js. The file that has been submitted is kept in the NPM registry.

The NPM registry is where developers can get the tools, they need to create software. Hence, before going on to use ReactJS, you should have a good grasp of the NPM (Node package manager) registry and how to use NPM to install packages.

3. Fundamentals of JavaScript and ES6

One of the most perplexing languages for programmers is JavaScript. You can't learn ReactJS without first learning JavaScript. React makes extensive use of JavaScript as compared to other frontend solutions. ECMAScript2015 or ES6 is a standard for scripting languages like JavaScript, ActionScript, and JScript. It was developed initially as a way to standardize JavaScript.

ES6 Features that you need to be familiarized

The following features of ES6 need to be familiarized to get a good grip of the language

Let and Const

Arrow functions

Default parameters

spread attributes

Maps

4. JavaScript XML(JSX) and Babel

JSX is HTML-flavored JavaScript. React elements are generated using JSX, which is a syntax extension of standard JavaScript. Hence, it is the best way to use HTML code in JavaScript.

Babel is a JavaScript compiler that converts HTML-like text in JavaScript files to standard JavaScript artifacts. It converts functionality from the most recent edition of JavaScript or [ECMAScript 2015+ \(ES6+\) to ES5](#). While using JSX is not needed by React, most people find it useful as a visual aid when working with UI in JavaScript code. React will display more useful error and alert messages.

The Difference Between a Framework and a Library

Link:

<https://www.freecodecamp.org/news/the-difference-between-a-framework-and-a-library-bd133054023f/>

Tools required to learn react

Vs code

Node js

Node js installation

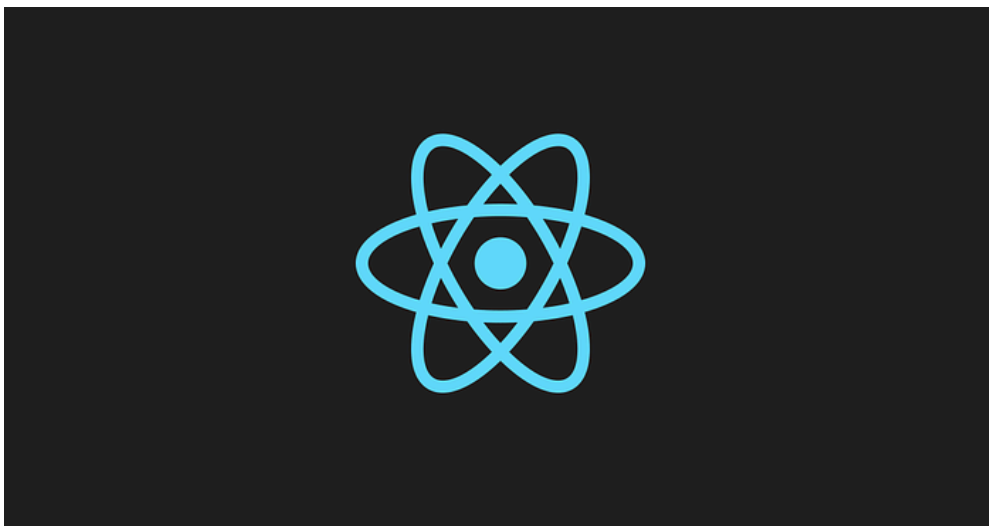
Download node js and install

ReactJS: A brief history

React is a popular free and open-source front end Javascript library that is used for building UI components and user interfaces.

It was created by Facebook and is still maintained by them along with an ever growing community of companies and individual developers.

React is commonly used as the code base for SPA's ([Single Page Applications](#)).



But how did React get its start?

In 2011, as Facebook's Ad app began to pick up major momentum, their code began seeing a new updates and team members which became overwhelming and hard to manage after a while. In an attempt to help remedy this problem, [Jordan Walke](#) created [FaxJS](#), a prototype which made this process more efficient. This was the birth of what would later evolve into the version of React that we all currently know and love today.

In 2012, Facebook bought a little company called Instagram, who had just 13 employees at the time, for [\\$1 Billion](#). After its

acquisition, Instagram wanted to adopt much of Facebook's new technology. This put pressure on the parent company to decouple React from Facebook and make it open-source, which is exactly what was done. In late May 2013, Jordan Walke finally introduced what was now called 'React' and open-sourced the material.

Some notable features which have helped make React so beloved by developers are its use of Components, the Virtual DOM, Lifecycle Methods, JSX, and React hooks.

React three main advantages which helped build and now sustain its immense success. Those three advantages are: *clean programming*, *fast performance*, and *a strong community*.

Some of the major companies that currently use React include: [Netflix](#), [Facebook](#), [Instagram](#), [Airbnb](#), [Reddit](#), [Dropbox](#), and [Postmates](#).

Setting Up React Environment – Step by step Guide

Features of react

React.js is a popular JavaScript library developed by Facebook for building user interfaces, especially single-page applications. It offers various features that make it a preferred choice for developers. Here are some of the key features of React.js:

1. Component-Based Architecture:

- React allows you to build encapsulated components that manage their own state, then compose them to create complex user interfaces.
- Components can be reused across different parts of an application, making the code more modular and maintainable.

2. Virtual DOM (Reconciliation:

- React uses a virtual DOM to enhance performance. It creates an in-memory data structure cache, computes the differences, and updates the DOM efficiently.
- This approach minimizes the number of manipulations to the real DOM, which is slower, resulting in faster and smoother UI updates.

3. JSX (JavaScript XML):

- JSX is a syntax extension that allows you to write HTML-like code within JavaScript. It makes the code more readable and expresses the structure of the UI more clearly.
- JSX compiles down to JavaScript, enabling the creation of React elements.

4. Unidirectional Data Flow:

- React enforces a one-way data flow, meaning data flows from parent to child components. This makes the app more predictable and easier to debug.
- The unidirectional flow ensures that changes in the state and data trigger re-rendering of the UI in a structured and controlled way.

5. State Management:

- React components can have internal states that can change over time. This allows for dynamic and interactive UIs.
- The state and lifecycle methods enable fine-grained control over the behavior of components.

6. Lifecycle Methods:

- React provides lifecycle methods that allow you to control what happens during different stages of a component's lifecycle (mounting, updating, and unmounting).
- Common methods include `componentDidMount`, `componentDidUpdate`, and `componentWillUnmount`.

7. Hooks:

- React Hooks, introduced in version 16.8, allow you to use state and other React features without writing a class.
- Common hooks include `useState` for managing state, `useEffect` for side effects, and `useContext` for context management.

8. Context API:

- The Context API provides a way to share data globally across components without passing props manually at every level.
- It's useful for managing global state like themes, authentication, or user settings.

9. React Router:

- React Router is a popular library for adding routing capabilities to React applications.
- It allows you to define multiple routes in your app, making it possible to build single-page applications with multiple views.

10. Performance Optimization:

- React offers tools like `React.memo`, `PureComponent`, and `shouldComponentUpdate` to optimize re-rendering and improve performance.
- You can also use techniques like lazy loading and code splitting with `React.lazy` and `Suspense` to load components on demand.

11. Developer Tools:

- React has a dedicated set of developer tools for Chrome and Firefox that allow you to inspect the component hierarchy, monitor props and state, and debug the application easily.

12. Ecosystem and Community Support:

- React has a vast ecosystem with a wealth of libraries and tools for state management, UI components, and more.
- The strong community support ensures regular updates, abundant tutorials, and quick help for any issues.

13. Server-Side Rendering (SSR):

- React can render on the server side using frameworks like Next.js, which can improve performance and SEO.
- SSR allows the initial page load to be faster and provides better search engine optimization (SEO).

14. Progressive Web App (PWA) Support:

- React can be used to build Progressive Web Apps, which provide offline capabilities and a native app-like experience.

15. Extensibility and Integration:

- React can be integrated with various libraries like Redux for state management, GraphQL for data fetching, and more, providing a highly flexible development experience.

React Environment with Vite – Step-by-Step Guide

Step 1: Install Node.js and NPM

Before creating a React project, ensure that **Node.js** and **NPM** are installed on your system.

- **Download:** Visit <https://nodejs.org> and download the **LTS version** (recommended).
- **Installation:**
 - Run the installer.
 - Accept the license agreement.
 - Choose installation path.
 - Ensure **Node.js** and **NPM** are added to **PATH**.
- **Verify installation:**

`node -v`

`npm -v`

✓ Step 2: Create a New React Project using Vite

Vite is a modern frontend build tool that offers a faster development experience than CRA.

- **Install Vite + React** (no global install needed):

`npm create vite@latest myapp`

- Replace myapp with your desired project name.
- Navigate into your project folder:

Project Name → myapp

Framework → React

Variant → JavaScript

`cd my-react-vite-app`

- Install dependencies:

npm install

`npm run dev`

✅ **First React App (Hello World with Vite)**

Let's build a simple **Hello World** app.

1. **Navigate to src/App.jsx.**
 2. Replace the content with:
 3. `import React from 'react';`
 - 4.
 5. `function App() {`
 6. `return (`
 7. `<div className="App">`
 8. `<h1>Hello World!</h1>`
 9. `</div>`
 10. `);`
 11. `}`
 - 12.
 13. `export default App;`
 14. Your browser will automatically refresh and display **"Hello World!"**.
-

React Errors – Fixing Common Issues (Vite Version)

If you run into package errors (e.g., ajv or module issues):

Step 1: Clean Up

```
rm -rf node_modules package-lock.json
npm cache clean --force
```

Step 2: Reinstall Dependencies

```
npm install
```

Step 3: Update Problematic Packages

```
npm list ajv ajv-keywords
npm install ajv@latest ajv-keywords@latest
npm update
```

Step 4: Add Resolutions (Optional – if issues persist)

In package.json:

```
"resolutions": {  
  "ajv": "6.12.6"  
}
```

Then run:

```
npx npm-force-resolutions  
npm install
```

Step 5: Downgrade ajv (if errors continue)

```
npm install ajv@6 ajv-keywords@latest
```

Step 6: Check for Duplicate ajv Installations

```
npm ls ajv
```

Detailed breakdown of the React + Vite project folder structure

React + Vite Project Folder Structure Explained

When you create a project with:

```
npm create vite@latest my-app -- --template react
```

You get this structure:

```
my-app/  
├── node_modules/  
├── public/  
├── src/  
│   ├── App.css  
│   ├── App.jsx  
│   ├── index.css  
│   └── main.jsx  
├── .gitignore  
├── index.html  
├── package.json  
├── vite.config.js  
└── README.md
```

Folder-by-Folder & File-by-File Explanation

Root Level Files & Folders

node_modules/

- Stores all installed npm packages
 - Auto-created after npm install
 - **Don't edit this folder!**
-

package.json

- Contains project metadata
- Lists all dependencies, scripts, and project name
- Example:

```
{
  "name": "my-app",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0"
  },
  ...
}
```

vite.config.js

- Vite's configuration file
- You can add plugins, aliases, or environment settings here
- Basic config:

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
});
```

index.html

- The main HTML file used in your app (not in /public)

- It contains the root `<div id="root">` where React renders
- Also links the entry script:

```
<body>
  <div id="root"></div>
  <script type="module" src="/src/main.jsx"></script>
</body>
```

✔ **Vite directly uses this file**, unlike CRA.

.gitignore

- Tells Git which files/folders to ignore
 - Common ones:
 - `node_modules/`
 - `dist/`
 - `.env`
-

README.md

- Documentation file for your project
 - Great for giving an overview to other devs
-
-

public/

- Stores **static assets** (images, logos, etc.)
 - Files here are served **as-is**
 - Accessed via URL paths like `/logo.png`
 - **Avoid placing sensitive files here**
-

src/

Where all your **React code** lives. Let's break it down:

main.jsx

- **Entry point** of your React app
- Renders `<App />` into the root element

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App.jsx';
import './index.css';
```

```
ReactDOM.createRoot(document.getElementById('root')).render(  
  <React.StrictMode>  
    <App />  
  </React.StrictMode>  
);
```

App.jsx

- The **root component** of your app
- Acts like a layout or router wrapper

```
function App() {  
  return <h1>Hello Vite + React</h1>;  
}
```

```
export default App;
```

App.css and index.css

- Optional CSS files
- App.css for styling specific to the App component
- index.css for global styles (like resets, fonts, etc.)

 You can replace with Tailwind, CSS Modules, or styled-components if needed.

You can add more folders to `src/`:

Folder	Purpose
components/ /	Reusable components (like Button, Card)
pages/	Page-level components (Home, About)
assets/	Images, fonts, icons
hooks/	Custom React hooks
context/	Context providers (auth, theme)
services/	API functions or data fetching

!



React Components

♦ What is a Component?

- A **component** is a reusable bit of code responsible for rendering **part of the user interface (UI)**.
 - It can be **as small as a button** or **as large as an entire page**.
 - **Every component returns a markup** (usually JSX in React).
-

♦ Types of Components

1. Functional Components

- These are JavaScript functions that return JSX.
- Simple and commonly used in modern React.

2. Class Components

- Use ES6 classes.
 - Useful when working with older codebases or needing state/lifecycle methods (though now mostly replaced by hooks).
-

♦ Common Component Examples

✚ UI Components – as small as

- Button
- List
- Image

📄 Page Components – as big as

- Header
 - Footer
 - Main Component
-

♦ Component Nesting

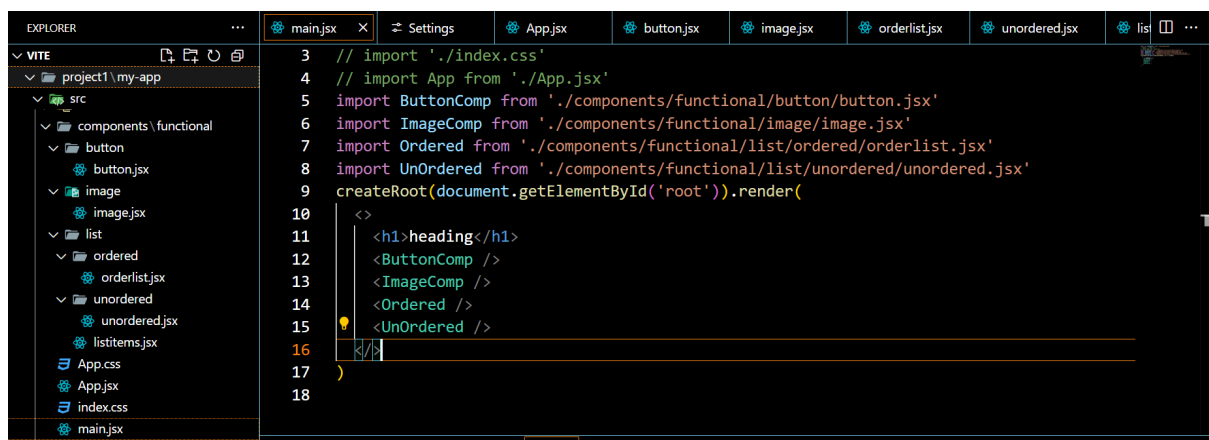
- You can **nest components** to build complex UIs.
- Example:
A Card component may include:
 - An Image
 - A Button
 - Some Text

🧩 Nesting Components

Components can include other components like LEGO blocks:

```
const Parent = () => {  
  return (  
    <div>  
      <Child1 />  
      <Child2 />  
    </div>  
  );  
};  
  
const Child1 = () => <p>I am Child 1</p>;  
const Child2 = () => <p>I am Child 2</p>;
```

Ex-1 – create a card using nested components



The screenshot shows a VS Code editor with a project named 'project1\my-app'. The Explorer sidebar on the left shows the file structure:

- src
 - components\functional
 - button
 - button.jsx
 - image
 - image.jsx
 - list
 - ordered
 - orderlist.jsx
 - unordered
 - unordered.jsx
 - listitems.jsx
 - App.css
 - App.jsx
 - index.css
 - main.jsx

The main editor shows the code in 'main.jsx':

```
3 // import './index.css'  
4 // import App from './App.jsx'  
5 import ButtonComp from './components/functional/button/button.jsx'  
6 import ImageComp from './components/functional/image/image.jsx'  
7 import Ordered from './components/functional/list/ordered/orderlist.jsx'  
8 import UnOrdered from './components/functional/list/unordered/unordered.jsx'  
9 createRoot(document.getElementById('root')).render(  
10   <>  
11     <h1>heading</h1>  
12     <ButtonComp />  
13     <ImageComp />  
14     <Ordered />  
15     <UnOrdered />  
16   </>  
17 )  
18
```


Output

heading

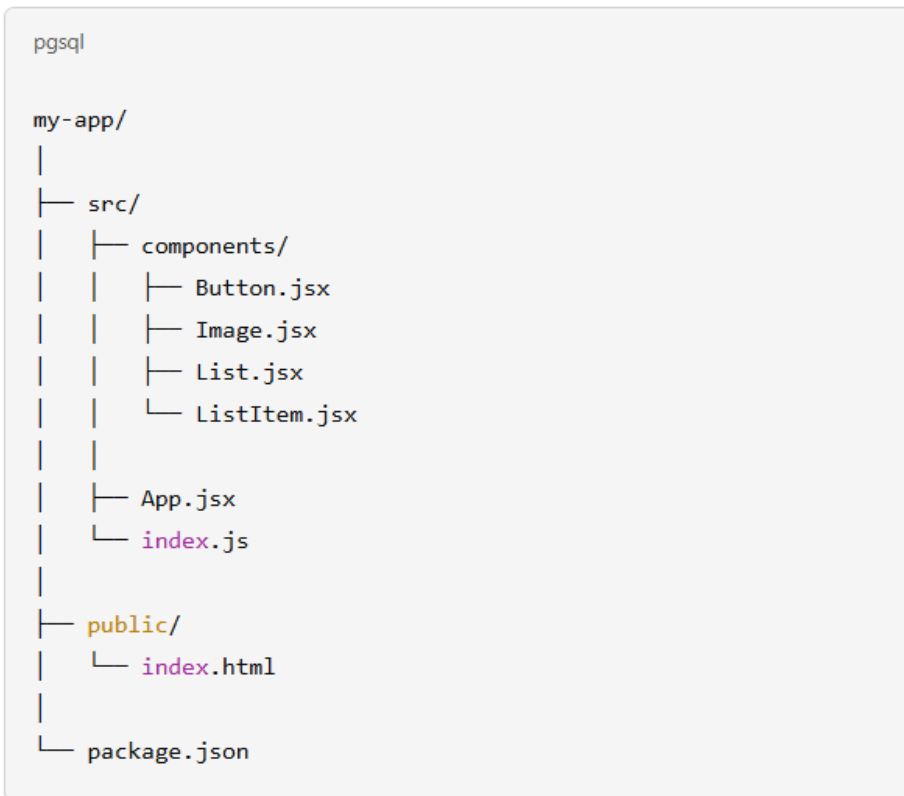


Login

1. apple
2. mango
3. orange

- apple
- mango
- orange

Folder Structure



Example of simple sub component

{ 4 return(5 6 <Listitems/> 7 8) 9 } 10 11 export default UnOrdered" data-bbox="119 518 880 739"/>

```
1 import Listitems from "../listitems"
2
3 const UnOrdered=()=>{
4     return(
5         <ul>
6             <Listitems/>
7         </ul>
8     )
9 }
10
11 export default UnOrdered
```

React Functional Components

What is a Functional Component?

A **functional component** is simply a **JavaScript function** that returns **JSX** (which looks like HTML). It is used to define a piece of the **UI (User Interface)**.

- ♦ **Basic Syntax:**

```
function MyComponent() {  
  return (  
    <div>  
      <h1>Hello from MyComponent</h1>  
    </div>  
  );  
}
```

or using ES6 arrow function:

```
const MyComponent = () => {  
  return (  
    <div>  
      <h1>Hello from MyComponent</h1>  
    </div>  
  );  
}
```

- ♦ **Rendering a Functional Component**

You **use** a component like an HTML tag (self-closing or with children):

```
<MyComponent />
```

You must include this inside a parent component or directly in the ReactDOM.render() like:

```
import { createRoot } from 'react-dom/client'  
import MyComponent from './App'  
  
createRoot(document.getElementById('root')).render(  
  <>  
    <MyComponent/>  
  </>  
)
```

```
</>  
)
```

Structure Example (Simple Page Layout):

```
const Header = () => {  
  return <h1>My Website</h1>;  
};  
  
const Footer = () => {  
  return <footer>© 2025 My Website</footer>;  
};  
  
const MainContent = () => {  
  return (  
    <div>  
      <p>This is the main content area.</p>  
    </div>  
  );  
};  
  
const App = () => {  
  return (  
    <div>  
      <Header />  
      <MainContent />  
      <Footer />  
    </div>  
  );  
};  
  
export default App;
```

App acts like the main/root component which uses smaller reusable components (Header, MainContent, Footer).

Organizing Functional Components

A common folder structure for functional components:

src/

```
|
|
| — components/
|   | — Header.js
|   | — Footer.js
|   | — MainContent.js
|
| — App.js
```

Each component file:

// Header.js

```
const Header = () => {
  return <h1>My Site</h1>;
};

export default Header;
```

In App.js:

```
import Header from './components/Header';

const App = () => {
  return <Header />;
};
```

🌟 JSX (JavaScript XML)

♦ What is JSX?

JSX stands for **JavaScript XML**. It is a **syntax extension** for JavaScript, commonly used with React to describe what the UI should look like. It looks like HTML but has the full power of JavaScript.

✅ Why Use JSX?

- Makes code **easier to write and understand**.
 - Looks like **HTML**, which is familiar to web developers.
 - Allows embedding **JavaScript expressions** within the markup.
 - JSX gets **compiled to `React.createElement()` calls** under the hood.
-

🔧 Babel

♦ What is Babel?

Babel is a **JavaScript compiler** that transforms modern JavaScript and JSX into code that browsers can understand.

🔧 JSX & Babel:

JSX is **not valid JavaScript**, so browsers can't read it directly. **Babel** converts JSX into plain JavaScript using the `React.createElement()` function.

💡 Example:

Input (JSX):

```
const greeting = <h1>Hello, JSX!</h1>;
```

Babel Output:

```
const greeting = React.createElement('h1', null, 'Hello, JSX!');
```

So, Babel + JSX = Modern and browser-friendly code.

📦 JSX Rules in React

JSX has **some strict rules and best practices**. Here's a full breakdown:

1. Return Only One Parent Element

A JSX expression must have **one root element**.

❌ Invalid:

```
const Parent = () => {
```

```
return (  
  <h1>Hello</h1>  
  <p>World</p>  
);  
};
```

✓ Valid:

```
const Parent = () => {  
  return (  
    <div>  
      <h1>Hello</h1>  
      <p>World</p>  
    </div>  
  );  
};
```

2. Use className Instead of class

class is a reserved keyword in JavaScript.

✗ Invalid:

```
<div class="box"></div>
```

✓ Valid:

```
<div className="box"></div>
```

3. Use camelCase for Attributes

HTML attributes are written in **camelCase** in JSX.

✓ Examples:

```
<input type="text" />  
<button onClick={handleClick}>Click</button>
```

4. Close All Tags

All elements must be **self-closed** if they don't have children.

✗ Invalid:

```

```

✓ Valid:

```

```

5. JSX Allows JavaScript Expressions Inside {}

You can embed any valid JS expression using curly braces {}.

```
const name = "React";
return (
  <div>
    <h1>Hello, {name}</h1>
  </div>
);
```

6. No if Statements Inside JSX

You can't use if inside JSX directly – instead use:

- **Ternary operator** condition ? true : false
- **Logical AND** condition && expression

✓ Examples:

```
{isLoggedIn ? <h1>Welcome</h1> : <h1>Please log in</h1>}
{showMessage && <p>This is visible only when showMessage is true</p>}
```

7. Fragments (<>...</>) to Avoid Extra Nodes

React components must return one root element, but you can use **fragments** if you don't want to add extra HTML elements.

```
<>
  <h1>Title</h1>
  <p>Description</p>
</>
```

8. Inline Styling in JSX

Use `style={{}}` with camelCase properties.

```
<div style={{ backgroundColor: 'skyblue', padding: '10px' }}>Hello</div>
```

9. Comments in JSX

Use curly braces with regular JS comments:

```
{/* This is a comment */}
```

Summary Table:

Concept	JSX Syntax	Notes
HTML in JS	<code><div>Hello</div></code>	Must return a single root
Attributes	<code>className</code> , <code>htmlFor</code> , <code>camelCase</code>	JSX uses JS naming conventions
JS expressions	<code>{expression}</code>	Inline JavaScript allowed
Conditional	<code>{condition ? A : B}</code> / <code>{condition && A}</code>	No if inside JSX
Styling	<code>style={{ color: 'red' }}</code>	CSS in JS object form
Comments	<code>{/* comment */}</code>	JSX comments

Modules in React (ES6 Modules)?

Modules are used to **split code** into multiple files for better organization, maintainability, and reusability. This modular approach is based on **ES6 module syntax**, allowing you to **export** and **import** code like components, functions, variables, etc.

Exporting in React (Two Types)

1. Default Export

- A file can have only one default export.

- It's imported **without curly braces**.

// Component1.js

```
const Component1 = () => {  
  return <h1>Component 1</h1>;  
};  
  
export default Component1;
```

2. Named Export

- A file can have **multiple named exports**.
- They are imported **with curly braces**.
- You can also use as to alias during import.

// Component2and3.js

```
export const Component2 = () => {  
  return <h1>Component 2</h1>;  
};  
  
export const Component3 = () => {  
  return <h1>Component 3</h1>;  
};
```

📦 Combining Default and Named Exports

You can combine both export types in a single file:

// export.js

```
const Component1 = () => {  
  return <h1>Component 1</h1>;  
};  
export default Component1;  
  
export const Component2 = () => {  
  return <h1>Component 2</h1>;  
};
```

```
export const Component3 = () => {  
  return <h1>Component 3</h1>;  
};
```

Importing Modules

Import Default Export

```
import Component1 from './export';
```

Import Named Exports

```
import { Component2, Component3 } from './export';
```

Import with Aliases (using as)

```
import A, { Component2 as B, Component3 as C } from './export';  
Now you can use:  
<A />  
<B />  
<C />
```

Why Use Modules in React?

1. **Separation of Concerns** – One file = one component.
2. **Code Reusability** – Easily reuse components/functions across files.
3. **Clean Structure** – Makes large applications manageable.
4. **Scoped Variables** – Variables in one module are not globally accessible.

Best Practices

- ✓ Use **default export** when the file contains only one main component.
 - ✓ Use **named exports** when you want to export multiple items.
 - ✓ Name files with the same name as the default export (e.g., Component1.js for Component1).
 - ✓ Group related components together in folders.
 - ✓ Avoid using both default and named exports in the same file unless necessary.
-

🧠 Real-World Example

Folder structure:

components/

├── Header.js

├── Footer.js

└── Layout.js

// Header.js

```
const Header = () => <header>Header</header>;  
export default Header;
```

// Footer.js

```
const Footer = () => <footer>Footer</footer>;  
export default Footer;
```

// Layout.js

```
import Header from './Header';  
import Footer from './Footer';  
  
const Layout = () => (  
  <div>  
    <Header />  
    <main>Content goes here</main>  
    <Footer />  
  </div>  
  
export default Layout;
```

React Conditional Rendering & Destructuring & Map method

✓ 1. Ternary Operator

Syntax:

condition ? statement1 : statement2

This helps choose between two elements or values.

♦ Example:

```
const ConditionalRender = () => {  
  const marks = 60;  
  
  return (  
    <div>  
      <h1>Student Name: John</h1>  
      {  
        marks > 90 ? <h5>A+</h5> :  
        marks > 80 ? <h5>A grade</h5> :  
        marks > 70 ? <h5>B grade</h5> :  
        <h5>Fail: Better luck next time</h5>  
      }  
    </div>  
  );  
};  
  
export default ConditionalRender;
```

Here we're using **nested ternary operators** to check multiple conditions.

✓ 2. Logical AND Operator (&&) - Short Circuit

Only renders the element if the condition is true.

Syntax:

condition && statement

♦ Example:

```
const ShowMessage = () => {  
  const isPassed = true;  
  
  return (  

```

```

    <div>
      <h1>Exam Result</h1>
      {isPassed && <p>Congratulations! You passed.</p>}
    </div>
  );
};

export default ShowMessage;

```

If isPassed is false, nothing will be shown.

✓ 3. Destructuring

You can destructure variables to use them easily inside your component.

♦ Example:

```

const StudentInfo = () => {
  const student = {
    name: "Alice",
    marks: 88
  };

  const { name, marks } = student; // destructuring

  return (
    <div>
      <h2>Name: {name}</h2>
      {
        marks > 90 ? <h4>A+</h4> :
        marks > 80 ? <h4>A grade</h4> :
        marks > 70 ? <h4>B grade</h4> :
        <h4>Fail</h4>
      }
    </div>
  );
};

export default StudentInfo;

```

✓ 4. Using map() with Conditional Rendering

If you have an array of data and want to render each item with conditions:

♦ Example:

```
const StudentList = () => {
  const students = [
    { name: "John", marks: 95 },
    { name: "Alice", marks: 78 },
    { name: "Bob", marks: 65 },
    { name: "Steve", marks: 50 }
  ];

  return (
    <div>
      <h2>Student Results</h2>
      {students.map((student, index) => (
        <div key={index}>
          <h4>{student.name}</h4>
          {
            student.marks > 90 ? <p>A+</p> :
            student.marks > 80 ? <p>A grade</p> :
            student.marks > 70 ? <p>B grade</p> :
            <p>Fail</p>
          }
        </div>
      ))}
    </div>
  );
};

export default StudentList;
```

✓ 5. Nested Conditional Rendering

You can nest conditional rendering using ternary and AND operators together.

♦ Example:

```
const NestedConditions = () => {
  const isLoggedIn = true;
  const marks = 85;

  return (
    <div>
      {
        isLoggedIn ? (
```

```

    <div>
      <h2>Welcome back!</h2>
      {
        marks > 90 ? <p>Your grade is A+</p> :
        marks > 80 ? <p>Your grade is A</p> :
        <p>Keep trying!</p>
      }
    </div>
  ) : (
    <h3>Please log in to see your result</h3>
  )
}
</div>
);
};
export default NestedConditions;

```

Summary

Concept	Use
Ternary Operator	To render one of two (or more) options
Logical AND (&&)	Render if condition is true
Destructuring	Cleanly extract data from objects
map() + condition	Render multiple elements with logic
Nested Conditional	Combine conditions inside each other

React Styling – Complete Notes

In React, there are **4 main ways to style components**:

1. **Inline Styles (Object Styles)**
 2. **External Stylesheets**
 3. **CSS Modules (Scoped Styles)**
 4. **Bootstrap**
-

1 Inline Styles (Object Styles)

✓ Description:

React allows you to add styles inline using **JavaScript objects**. This is useful for quick styles or dynamic styles.

✓ Syntax:

- Property names are **camelCased**
- Values are strings (except for numeric values that don't require units like px)

```
function MyComponent() {  
  const headingStyle = {  
    color: 'blue',  
    fontSize: '24px',  
    textAlign: 'center',  
  };  
  
  return <h1 style={headingStyle}>Hello, Inline Styling!</h1>;  
}
```

✓ Pros:

- Good for dynamic styles
- Quick styling for small components

✗ Cons:

- No pseudo selectors (like :hover)
- No media queries
- Not reusable

2 External Stylesheets

✓ Description:

You can write regular CSS in .css files and import them into your React component or root file.

✓ Steps:

1. Create a CSS file (App.css)
2. Import it inside the React file using import './App.css'

/* App.css */

```
.title {
```

```
color: green;
font-size: 30px;
text-align: center;
}
```

```
import './App.css';

function App() {
  return <h1 className="title">External Styling</h1>;
}
```

✓ Pros:

- Supports media queries, pseudo selectors
- Reusable and maintainable

✗ Cons:

- Global scope (styles may leak between components)

3 CSS Modules (Scoped Styles)

✓ Description:

CSS Modules are CSS files in which all class and animation names are scoped locally by default.

✓ Steps:

1. Name your file as ComponentName.module.css
2. Import it into your component
3. Use styles.className syntax

```
/* MyComponent.module.css */
```

```
.title {
  color: green;
  font-size: 30px;
  text-align: center;
}
```

```
import styles from './MyComponent.module.css';
```

```
function MyComponent() {  
  return <h1 className={styles.title}>CSS Module Styling</h1>;  
}
```

✓ Pros:

- Locally scoped by default (no style leakage)
- Good for large apps with many components

✗ Cons:

- Slightly more complex syntax
 - Not supported outside of build environment (like raw HTML)
-

4. Bootstrap Styling

✓ Description:

Bootstrap is a popular CSS framework. You can use it in two ways:

✓ Method 1: Using CDN

1. Add Bootstrap CDN link in public/index.html

<!-- Inside public/index.html -->

```
<link  
  rel="stylesheet"  
  
  href="https://maxcdn.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.css"  
>
```

2. Use Bootstrap classes in JSX:

```
function App() {  
  return <button className="btn btn-primary">Click Me</button>;  
}
```

✓ Method 2: Using NPM Install

1. Install Bootstrap:

```
npm install bootstrap
```

2. Import Bootstrap CSS in index.js or App.js:

```
import 'bootstrap/dist/css/bootstrap.min.css';
```

3. Use classes as before:

```
<button className="btn btn-success">Click Me</button>
```

✓ Pros of UI Libraries:

- Pre-designed components (buttons, modals, forms)
- Faster development
- Consistent UI design

✗ Cons:

- Less control over design
- Slightly larger bundle size

🔍 Summary Table

Style Type	Scope	Supports Pseudo Selectors	Reusability	Use Case
Inline Styles	Component	✗	✗	Quick, dynamic styles
External Stylesheet	Global	✓	✓	App-wide styles
CSS Modules	Component	✓	✓	Scoped styles per component
SASS	Component	✓	✓	Structured, scalable styles
Bootstrap (CDN/NPM)	Global/Scoped	✓	✓	Prebuilt responsive UI components

Presentation Layer and Container Layer in React

✂ What is the Presentation Layer?

- Also called **presentational** or **dumb** components.
- Responsible **only for UI** – how things look.
- It **receives data via props** and displays it.
- Does **not manage state** or data fetching.

✅ Example: Presentation Component

// UserGreeting.js

```
function UserGreeting({ name }) {  
  return <h2>Hello, {name}!</h2>;  
}  
  
export default UserGreeting;
```

🔧 What is the Container Layer?

- Also called **container** or **smart** components.
- Responsible for **logic, data, and state management**.
- It **fetches or stores data**, and passes it to the presentation component via props.

✅ Example: Container Component

// UserContainer.js

```
import React from 'react';  
import UserGreeting from './UserGreeting';  
  
function UserContainer() {  
  // Logic / data (could be from API or state)  
  const user = { name: "Teja" };  
  
  return (  
    <div>  
      <h1>Welcome Section</h1>  
      <UserGreeting name={user.name} />  
    </div>  
  );  
}
```

```
}  
  
export default UserContainer;
```

📌 Summary Table

Layer	Responsibility	Manages State	Renders UI	Reusability
Presentation	Displays UI only	❌ No	✅ Yes	✅ High
Container	Handles logic & data	✅ Yes	✅ Yes	⚠️ Lower

Props in React?

📦 What are Props in React?

Props (short for **Properties**) are **read-only inputs** to React components. They allow you to **pass data from a parent component to a child component**, making components reusable and dynamic.

🧠 Why use Props?

- To make components **reusable** and **dynamic**.
- To allow **parent-child communication**.
- To separate **logic from presentation**.
- To share **data between components** in a one-way direction (**unidirectional data flow**).

📌 Key Characteristics of Props

Feature	Description
Read-only	Cannot be modified by the child component
Passed by parent	Always passed from a parent to a child component
JavaScript objects	Props are just regular JavaScript objects

Feature	Description
Unidirectional	Data flows from parent to child, not vice versa

✓ Basic Example of Props

```
// Parent Component
function App() {
  return <Greeting name="Teja" />;
}

// Child Component
function Greeting(props) {
  return <h1>Hello, {props.name}!</h1>;
}
```

Output: Hello, Teja!

Here, name="Teja" is a prop passed from App to Greeting.

🔧 Destructuring Props

To write cleaner code, you can destructure props like this:

```
function Greeting({ name }) {
  return <h1>Hello, {name}!</h1>;
}
```

🎯 Passing Multiple Props

```
function Profile({ name, age }) {
  return (
    <div>
      <p>Name: {name}</p>
      <p>Age: {age}</p>
    </div>
  );
}

// Usage
<Profile name="Teja" age={22} />
```

Reusability with Props

You can reuse components with different values:

```
<Greeting name="Teja" />
<Greeting name="John" />
<Greeting name="Aisha" />
```

Things to Remember

- Props are **immutable** inside the child.
- If a child tries to modify a prop, it won't affect the parent.
- Use **state** (not props) when you need to manage internal values that change.

Real-Life Analogy

Think of props like function parameters. Just like you call a function with arguments, you call a component with props.

Summary

Concept Description

What	Inputs to components
Direction	One-way (parent → child)
Mutability	Read-only
Usage	Customize component behavior
Benefits	Reusability, cleaner UI logic, better structure

◆ Understanding Props with Your Example

You have a component tree like this:

```
AccordionComp (parent)
├─ AccordionItem (child)
│   └─ Cardcom (child of AccordionItem)
```

Let's walk through how props are used at each step.


```

import Accordion from 'react-bootstrap/Accordion';
import Cardcom from './card';

function AccordionComp() {
  const products = [
    {
      "id": 1,
      "title": "Fjallraven - Foldsack No. 1 Backpack, Fits 15 Laptops",
      "price": 109.95,
      "description": "Your perfect pack for everyday use and walks in the forest. Stash your laptop (up to 15 inches) in the padded sleeve, your everyday",
      "category": "men's clothing",
      "image": "https://fakestoreapi.com/img/81fPKd-2AYL._AC_SL1500_.jpg",
      "rating": {
        "rate": 3.9,
        "count": 120
      }
    },
    {
      "id": 2,
      "title": "Mens Casual Premium Slim Fit T-Shirts ",
      "price": 22.3,
      "description": "Slim-fitting style, contrast raglan long sleeve, three-button henley placket, light weight & soft fabric for breathable and comfortable wearing. And Solid stitched shirts with round neck made for durability and a great fit for casual fashion wear and diehard baseball fans. The Henley style round neckline includes a three-button placket.",
      "category": "men's clothing",
      "image": "https://fakestoreapi.com/img/71-3HjGNDUL._AC_SY879._SX_UX._SY._UY_.jpg",
      "rating": {
        "rate": 4.1,
        "count": 259
      }
    },
    {
      "id": 3,
      "title": "Mens Cotton Jacket",
      "price": 55.99,
      "description": "great outerwear jackets for Spring/Autumn/Winter, suitable for many occasions, such as working, hiking, camping, mountain/rock climbing, cycling, traveling or other outdoors. Good gift choice for you or your family member. A warm hearted love to Father, husband or son in this thanksgiving or Christmas Day.",
      "category": "men's clothing",
      "image": "https://fakestoreapi.com/img/71li-ujtUL._AC_UX679_.jpg",
      "rating": {
        "rate": 4.7,
        "count": 500
      }
    }
  ]
  return (
    <>
    <Accordion defaultActiveKey="0" >
      {
        products.map((eachObject, ind) => {
          return <>
            <AccordionItem data={eachObject} key={ind} />
          </>
        })
      }
    </Accordion>
    </>
  )
}

```

```

    );
  }

export default AccordionComp;

function AccordionItem(props) {
  return (
    <Accordion.Item eventKey={props.data.id} key={props.data.id} >
      <Accordion.Header>Accordion Item
{props.data.id}</Accordion.Header>
      <Accordion.Body>
        {props.data.title}
      </Accordion.Body>
      <Accordion.Body>
        <Cardcom data={props.data} />
      </Accordion.Body>
    </Accordion.Item>
  )
}

```

```

import Button from 'react-bootstrap/Button';
import Card from 'react-bootstrap/Card';

function Cardcom(props) {
  console.log(props)
  const {title,description,image}=props.data

  return (
    <Card style={{ width: '18rem' }}>
      <Card.Img variant="top" src={image}/>
      <Card.Body>
        <Card.Title>{title}</Card.Title>
        <Card.Text>
          {description}
        </Card.Text>
        <Button variant="primary">Go somewhere</Button>
      </Card.Body>
    </Card>
  );
}

```

```
export default Cardcom;
```

✓ 1. Passing props from AccordionComp to AccordionItem

```
<AccordionItem data={eachObject} key={ind} />
```

- Here, the AccordionComp has a products array.
- It loops through the array using .map() and sends each product (eachObject) as a **prop** to AccordionItem.
- data={eachObject} → this is how the individual product object is passed as a prop.

✓ 2. Receiving props in AccordionItem

```
function AccordionItem(props) {  
  return (  
    <Accordion.Item eventKey={props.data.id} key={props.data.id}>  
      <Accordion.Header>Accordion Item  
{props.data.id}</Accordion.Header>  
      <Accordion.Body>  
        {props.data.title}  
      </Accordion.Body>  
      <Accordion.Body>  
        <Cardcom data={props.data} />  
      </Accordion.Body>  
    </Accordion.Item>  
  );  
}
```

- The prop is received as props object in the function parameter.
- props.data gives access to the full product object (like title, description, image, etc.)
- The same product data is further **passed down to Cardcom** using:

- <Cardcom data={props.data} />

This is called **prop drilling**: sending data down multiple levels.

✓ 3. Receiving props in Cardcom

```
import Button from 'react-bootstrap/Button';
import Card from 'react-bootstrap/Card';

function Cardcom(props) {
  console.log(props)
  const {title,description,image}=props.data

  return (
    <Card style={{ width: '18rem' }}>
      <Card.Img variant="top" src={image}/>
      <Card.Body>
        <Card.Title>{title}</Card.Title>
        <Card.Text>
          {description}
        </Card.Text>
        <Button variant="primary">Go somewhere</Button>
      </Card.Body>
    </Card>
  );
}

export default Cardcom;
```

- Cardcom receives the data via props, and you are destructuring it for cleaner code.
- Then these values are used to populate the Card component.

🔍 Console Logging props (Debugging)

```
console.log(props)
```

You are logging props inside Cardcom, which helps verify what data was received from the parent. It's useful for **debugging**.

🎯 Why Props Are Important?

1. **Component Reusability**
Same Cardcom can be used for any product by just changing the props.
2. **Separation of Concerns**
One component fetches/manages the data, another just renders it.

3. Clean Code

No need to hard-code values; components stay dynamic and flexible.

📌 Summary of Props Usage in Your Code

Component	Prop Passed	Purpose
AccordionItem	<code>data={eachObject}</code>	To pass product info to be displayed
Cardcom	<code>data={props.data}</code>	To show image, title, description

🧠 Best Practices

- Always use `key` while rendering lists with `.map()` to avoid React warnings.
 - Use **destructuring** in components for cleaner and readable code.
 - Props should be considered **immutable** inside child components.
-

✨ Final Output (Visual Flow)

AccordionComp (data loop)

- └─ AccordionItem (receives one product)
 - └─ Displays title in header
 - └─ Renders `<Cardcom data={product}/>`
 - └─ Shows image, title, description in a Bootstrap Card

👤 React children Prop

✅ What is the children Prop?

In React, **children** is a special prop that is automatically passed to a component when you put **JSX elements inside its opening and closing tags**.

♦ Example:

```
<MyComponent>
  <p>This is a child element.</p>
</MyComponent>
```

Here, `<p>...</p>` is **not passed as a regular prop**, but instead it is available inside `MyComponent` as `props.children`.

✓ Basic Usage

♦ Parent Component:

```
function Parent() {  
  return (  
    <Wrapper>  
      <h1>Hello from Parent</h1>  
      <p>This is inside the Wrapper component.</p>  
    </Wrapper>  
  );  
}
```

♦ Child Component (Wrapper):

```
function Wrapper(props) {  
  return (  
    <div style={{ border: '2px solid blue', padding: '10px' }}>  
      {props.children}  
    </div>  
  );  
}
```

♦ Output:

```
+-----+  
| Hello from Parent          |  
| This is inside the Wrapper... |  
+-----+
```

✨ props.children allows the Wrapper component to be flexible and reusable.

✓ Destructuring children for Clean Code

Instead of writing props.children, you can **destructure** directly:

```
function Wrapper({ children }) {  
  return (  
    <div className="box">  
      {children}  
    </div>  
  );  
}
```

```
);  
}
```

✓ Practical Example: Card Component

♦ Card.js

```
function Card({ children }) {  
  return (  
    <div style={{  
      border: '1px solid #ddd',  
      borderRadius: '8px',  
      padding: '16px',  
      margin: '10px',  
      boxShadow: '2px 2px 12px rgba(0,0,0,0.1)'  
    }}>  
      {children}  
    </div>  
  );  
}  
  
export default Card;
```

♦ App.js

```
import Card from './Card';  
  
function App() {  
  return (  
    <div>  
      <Card>  
        <h2>User Profile</h2>  
        <p>Name: Teja</p>  
        <p>Role: Developer</p>  
      </Card>  
  
      <Card>  
        <h2>Notification</h2>  
        <p>You have 3 new messages.</p>  
      </Card>  
    </div>  
  );  
}
```

```
    </div>
  );
}
```

With this pattern, Card becomes a **generic container** that can wrap any content.

✓ Summary

Feature	Description
children prop	Built-in prop that holds nested content
Use case	Reusable wrappers, layout components, modal, card, etc.
Accepts	JSX, strings, elements, arrays of elements, null, etc.
Tools	Use React.Children for safe handling
Benefit	High flexibility and reusability in component design

React Fragments

✓ What is a React Fragment?

A **React Fragment** lets you **group multiple elements** without adding extra nodes (like `<div>`) to the DOM.

Why is this useful?

Sometimes, you need to return **multiple elements from a component**—but JSX requires one root element.

Instead of this:

```
return (
  <div>
    <h1>Hello</h1>
    <p>Welcome</p>
  </div>
);
```

You can do this:

```
return (
  <>
```



```
<h1>Hello</h1>
<p>Welcome</p>
</>
);
```

✓ Syntax of Fragments

♦ Full Syntax (Using React.Fragment):

```
return (
  <React.Fragment>
    <h2>Heading</h2>
    <p>This is a paragraph.</p>
  </React.Fragment>
);
```

♦ Short Syntax (Empty angle brackets <>):

```
return (
  <>
    <h2>Heading</h2>
    <p>This is a paragraph.</p>
  </>
);
```

- ♦ Short syntax can't accept props (like key), but it's more concise.

✓ Why Use Fragments?

♦ 1. Avoid Extra Nodes in the DOM

Extra <div>s can mess up layout or styling. Fragments solve this.

```
// BAD: Adds unnecessary div
<div>
  <td>Row 1</td>
  <td>Row 2</td>
</div>
```

```
// GOOD: Using Fragment
<React.Fragment>
  <td>Row 1</td>
  <td>Row 2</td>
</React.Fragment>
```

♦ 2. Cleaner Markup

No need for wrapper divs → simpler HTML output.

✅ When Do You NEED React.Fragment (Full Syntax)?

When you want to:

- Add **props** to the fragment (e.g., key).
- Use **mapping** and return multiple elements **without wrapper divs**.

Example with key:

```
const items = ['Apple', 'Banana', 'Cherry'];

return items.map((item, index) => (
  <React.Fragment key={index}>
    <dt>{item}</dt>
    <dd>Description of {item}</dd>
  </React.Fragment>
));
```

- ♦ You can't use the short syntax <> when adding key or other props.
-

✅ Practical Example

Without Fragment (Unnecessary divs)

```
function Example() {
  return (
    <div>
      <h1>Title</h1>
      <p>Content</p>
    </div>
```

```
);  
}
```

This adds a <div> to the DOM which may not be needed.

With Fragment

```
function Example() {  
  return (  
    <>  
      <h1>Title</h1>  
      <p>Content</p>  
    </>  
  );  
}
```

Cleaner DOM, no extra tags added.

✅ When to Use Fragments

Use Case	Fragment?
Group multiple JSX elements	✅
Avoid extra HTML wrappers	✅
Returning elements in a map loop	✅
Need to pass props like key	Use React.Fragment
Returning one root element	❌ (no need)

✅ Summary

Feature	Description
React Fragment	Groups elements without adding extra DOM nodes
Short syntax	<>...</> for simplicity
Full syntax	<React.Fragment> allows adding props like key
Benefits	Cleaner HTML, better layout, avoids unnecessary wrappers

Feature	Description
Common use cases	Table rows, list rendering, layout components

♦ What is state in React (Class Components)?

- state is a built-in object in React used to store **dynamic data** or **component-specific information**.
 - It is **private** to the component and **mutable** (unlike props).
 - State determines the **render output** and **behavior** of the component.
 - When state is updated using `setState()`, the component **automatically re-renders** to reflect the new data on the UI.
-

♦ Key Features of state:

- Holds data that **may change over time**.
 - **Mutable** → You can change the value using `setState()`.
 - Changes in state cause **component re-render**.
 - Should **never be modified directly** (i.e., avoid `this.state.count = 1`)—always use `setState()`.
 - Used only in **class components** (in functional components, we use the `useState` hook).
-

♦ Creating State in Class Components

There are **two common ways** to create state:

1. Object Property Outside Constructor (Less Common)

```
state = {  
  count: 0  
}
```

2. Using Constructor Method (Recommended)

The `constructor()` method is used to:

- Initialize the state
 - Call the parent class (Component) constructor using `super()`
-

```
constructor() {  
  super(); // Calls the parent constructor  
  this.state = {  
    count: 0  
  };  
}
```

- this refers to the current class instance.
- super() is necessary when using constructor to access this.

♦ What is setState()?

- setState() is the method used to **update the component's state**.
- It causes the component to **re-render**.
- It is **asynchronous** – meaning it doesn't immediately update the state.

Syntax:

```
this.setState(updater, callback);
```

- updater: An object or function to update the state.
- callback: (Optional) A function that runs **after** state has been updated.

♦ Why use the callback inside setState()?

Because setState is asynchronous, if you try to access the updated state **immediately after setState**, you'll get the **old value**.

```
this.setState({ count: this.state.count + 1 }, () => {  
  console.log("Updated count:", this.state.count);  
});
```

Without the callback:

```
this.setState({ count: this.state.count + 1 }, () => {  
  console.log("Updated count:", this.state.count);  
});
```

💡 Using a Function in setState

Useful when new state depends on previous state:

```
this.setState((prevState) => ({  
  count: prevState.count + 1  
}));
```

♦ Full Example: Counter Component

```
import { Component } from "react";  
  
class Countercomp extends Component {  
  
  constructor() {  
    super();// Initialize base class  
    this.state = {  
      count: 0  
    };  
  }  
  
  // Arrow functions auto-bind 'this'  
  // + Increment button  
  increment = () => {  
    this.setState({ count: this.state.count + 1 }, () => {  
      console.log('Count updated to: ${this.state.count}');  
    });  
  
    console.log(this.state.count); // Will log old value  
  };  
  
  // - Decrement button  
  decrement = () => {  
    this.setState({ count: this.state.count - 1 }, () => {  
      console.log('Count updated to: ${this.state.count}');  
    });  
  };  
  
  // 🔄 Reset to 0  
  reset = () => {  
    this.setState({ count: 0 });  
  };  
}
```

```

};

// ▶▶ Step Increment: Add specific number
step = (val) => {
  this.setState({ count: this.state.count + val });
};

render() {
  console.log("component rerendering");
  return (
    <>
      <h3>Hello Counter</h3>
      <h4>{this.state.count}</h4>

      <button onClick={this.increment}>Increment</button>
      <button onClick={this.decrement}>Decrement</button>
      <button onClick={this.reset}>Reset</button>

      { /* Passing argument */ }
      <button onClick={() => this.step(3)}>Step Increment</button>
    </>
  );
}
}

export default Countercomp;

```

♦ Notes on Method Binding:

- Arrow functions (increment = () => {}) are used to **automatically bind this** to the class instance.
 - Without arrow functions, you'd need to bind methods manually in the constructor:
 - this.increment = this.increment.bind(this);
-

♦ Summary:

Concept	Description
state	Object to store dynamic data
setState()	Method to update state

Concept	Description
constructor()	Initializes the state using super()
this	Refers to current component instance
Callback in setState	Ensures you get updated state
Re-rendering	Happens automatically after state change

React Class Component – Subscription Logic Explained

What This Logic Does

The subscription logic allows the user to:

- **Toggle between "Subscribe" and "Unsubscribe"**
- **Conditionally display content** (like the counter), only if the user is subscribed

Step 1: Initial State

```
this.state = {  
  isSubscribed: false  
};
```

Explanation:

- isSubscribed is a **Boolean** (true or false)
- Initially set to false, meaning the user is **not subscribed**

Step 2: Toggle Function – changeviewer()

```
changeviewer = () => {  
  this.setState({  
    isSubscribed: !this.state.isSubscribed  
  });  
}
```

What's Happening Here?

- !this.state.isSubscribed means:
 - if it's false, change to true
 - if it's true, change to false

This is called **Boolean toggling** — flipping the value back and forth.

🎨 Step 3: Button UI

```
<button onClick={this.changeviewer}>
  { this.state.isSubscribed ? "unsubscribe" : "subscribe" }
</button>
```

🔍 Explanation:

- When clicked, changeviewer() is triggered to **toggle the state**
- The button **label text changes dynamically**:
 - isSubscribed = false → shows "subscribe"
 - isSubscribed = true → shows "unsubscribe"

This is done using a **ternary operator**:

condition ? "If True" : "If False"

👁️ Step 4: Conditional Rendering Based on Subscription

```
{
  this.state.isSubscribed ? (
    <>
      /* counter UI goes here */
    </>
  ) : (
    "please subscribe to access"
  )
}
```

🔍 Explanation:

- If isSubscribed === true
 - Shows the content inside <></> (like counter)
- If isSubscribed === false
 - Shows: "please subscribe to access"

✅ This is a common way to **protect or hide content** until the user meets a condition.

🧠 Summary Table

Concept	Description
isSubscribed	Boolean state to track subscription
!this.state.isSubscribed	Flips the Boolean value
Ternary operator	condition ? trueResult : falseResult
Conditional rendering	Shows different UI depending on state

✅ Key Takeaways

- You can **toggle boolean values** in state using !stateValue
 - You can **dynamically change text and content** using conditional logic in JSX
 - setState() updates component state and causes **re-render**
 - This logic is very useful for **login systems, visibility toggles, premium feature locks**, etc.
-

🔒 React Login/Signup Toggle (Class Component)

```
import { Component } from "react";

class AuthComponent extends Component {
  constructor() {
    super();
    this.state = {
      isLoggedIn: true, // true = Login, false = Signup
    };
  }

  toggleAuth = () => {
    this.setState({ isLoggedIn: !this.state.isLoggedIn });
  };

  render() {
    return (
      <div style={{ textAlign: "center", padding: "20px" }}>
        <h2>{this.state.isLoggedIn ? "Login Page" : "Signup Page"}</h2>

        {/* Form UI */}
      </div>
    );
  }
}
```

```

{
  this.state.isLogin ? (
    <>
      <input type="text" placeholder="Email" /><br />
      <input type="password" placeholder="Password" /><br />
      <button>Login</button>
    </>
  ) : (
    <>
      <input type="text" placeholder="Name" /><br />
      <input type="text" placeholder="Email" /><br />
      <input type="password" placeholder="Password" /><br />
      <button>Signup</button>
    </>
  )
}

<br /><br />
<button onClick={this.toggleAuth}>
  {this.state.isLogin ? "Switch to Signup" : "Switch to Login"}
</button>
</div>
);
}
}

export default AuthComponent;

```

📌 How It Works

- isLogin: tracks whether the user is on Login (true) or Signup (false)
- toggleAuth(): toggles the value of isLogin
- Uses **ternary operators** to:
 - Change **heading**
 - Show **different form fields**
 - Change **button label**

💡 Output

When isLogin = true

✅ Shows: Email + Password fields + Login button

When isLogin = false

✓ Shows: Name + Email + Password + Signup button

And a toggle button like:

👉 "Switch to Signup" or "Switch to Login"

React Class Component – Sorting Product Cards

✓ What it does:

- Displays a list of product cards (title + price)
 - Sorts them by **price (low to high)** when button is clicked
-

🔧 Key Parts:

1. State Setup

```
this.state = {  
  products: [/* product objects */]  
};
```

2. Sorting Function

```
filteredstate = () => {  
  this.setState({  
    products: this.state.products.sort((a, b) => a.price - b.price)  
  });  
}
```

Sorts products by ascending price and updates state

3. UI

```
<button onClick={this.filteredstate}>Sort by Price</button>  
  
{this.state.products.map(val => (  
  <div key={val.id}>  
    <h3>{val.title}</h3>  
    <h6>{val.price}</h6>  
  </div>  
))}
```

Maps through sorted products and displays them

React Class Components: state vs props

✓ state in Class Components

- **Definition:**
state is a built-in object in React components that is **used to store dynamic data**. It determines the component's behavior and how it renders.
- **Characteristics:**
 - state is **private** and fully controlled by the component itself.
 - It is **mutable** (you can change it using `this.setState()`).
 - It is initialized in the constructor using `this.state = {}`.
- **Syntax:**

```
class MyComponent extends React.Component {  
  constructor() {  
    super();  
    this.state = {  
      count: 0  
    };  
  }  
  
  render() {  
    return <h1>{this.state.count}</h1>;  
  }  
}
```

- **Update state:**

```
this.setState({ count: this.state.count + 1 });
```

✓ props in Class Components

- **Definition:**
props (short for properties) are **read-only** values **passed from a parent component to a child component**. They allow **reusability** and **data flow** between components.
- **Characteristics:**
 - props are **immutable** – you **cannot modify** them.
 - Passed to the component like attributes in HTML.
 - Can be accessed in class components using `this.props`.
- **Syntax (Passing props):**

```
<ChildComponent name="Teja" />
```

- **Access props:**

```
class ChildComponent extends React.Component {  
  render() {  
    return <h2>Hello, {this.props.name}</h2>;  
  }  
}
```

Summary Table

Feature	state	props
Ownership	Private to the component	Passed by parent
Mutability	Mutable	Immutable
Access Syntax	this.state.key	this.props.key
Purpose	Store local component data	Receive external data
Modifiable by	Component itself	Parent component

React - DOM Concepts: Real DOM vs Virtual DOM, Diffing & Reconciliation

♦ 1. Real DOM (Document Object Model)

Definition:

The Real DOM is the actual structured representation of the webpage loaded in the browser. It is an interface that allows scripts to update the content, structure, and style of a document.

How It Works:

- Any change (like updating a button label) involves re-rendering the whole UI.
- Manipulations are **directly reflected** on the webpage.
- It is **slow and inefficient** when dealing with frequent updates.

Drawbacks:

- **Performance issues:** Updating the real DOM frequently can be slow.

- **Re-rendering is expensive:** The whole tree (or large parts of it) needs to be updated even for small changes.
 - **Less optimized for modern dynamic UIs.**
-

♦ 2. Virtual DOM

Definition:

A **Virtual DOM** is an in-memory lightweight representation of the real DOM created by React.

How It Works:

- React builds a **Virtual DOM tree** whenever the UI changes.
- It compares the new Virtual DOM with the previous one using **Diffing**.
- Only the changes are then applied to the real DOM.

Benefits:

- **Faster updates:** Reduces the number of direct DOM manipulations.
 - **Efficient rendering:** React calculates the minimal set of changes.
 - **Better user experience:** Less flickering, more responsive UIs.
-

3. Diffing

Definition:

Diffing is the process of comparing two versions of the Virtual DOM (old vs new) to determine what has changed.

How It Works:

- React uses an optimized **Diffing algorithm** to detect differences.
- It identifies which elements have changed, been added, or removed.
- React then creates a **patch** of those changes.

Why It's Efficient:

- **No full re-rendering:** Only parts that changed are updated.
 - **Faster than Real DOM manipulation.**
 - Reduces rendering time and boosts performance.
-

4. Reconciliation

Definition:

Reconciliation is the process where React updates the real DOM with the changes identified during diffing.

⚙️ How It Works:

- React applies the minimum number of changes necessary to the real DOM.
- This makes the update process fast and efficient.

🔧 Key Features:

- **Selective rendering:** Only the modified components are re-rendered.
 - **Performance optimization:** React skips unnecessary updates.
 - **Maintains UI consistency** with minimal overhead.
-

! Limitations of React

Although React is powerful and widely used, it comes with a few **limitations and trade-offs** that developers should be aware of:

♦ 1. React is a Library, Not a Full Framework

- React focuses **only on the view layer (UI)** of the application (the "V" in MVC).
 - **Limitation:** It doesn't include built-in solutions for routing, form handling, HTTP requests, or state management.
 - **Result:** Developers often need to rely on **third-party libraries** like:
 - React Router (for routing)
 - Redux / Zustand / Recoil (for state management)
 - Axios or Fetch (for API calls)
 - Formik or React Hook Form (for form handling)
-

♦ 2. Too Many Choices (Ecosystem Complexity)

- The flexibility of choosing your tools can become **overwhelming**, especially for beginners.
 - There's **no "official" way** to build things beyond UI, which leads to inconsistent architectures in teams.
 - Developers often spend time choosing, comparing, and integrating third-party tools.
-

♦ 3. JSX as a Barrier for New Developers

- React uses JSX (JavaScript + XML), which is not pure JavaScript.
 - JSX may **confuse beginners** who are new to web development or come from non-JavaScript backgrounds.
 - It mixes HTML and JavaScript, which some developers find **unintuitive or hard to debug**.
-

♦ 4. React Is UI-Centric (Not Suitable for All Applications)

- React excels at building smooth, interactive UIs.
 - But it may **not be the best fit** for:
 - **Banking applications**
 - **Financial software**
 - Systems requiring high security, strict regulation, and real-time data accuracy.
 - These systems often need **fully server-controlled rendering** and **strong typing**, which are better supported in other frameworks like Angular or backend-driven architectures.
-

♦ 5. Performance Issues in Large Applications

- While React is optimized, **improper use** of components or state can lead to:
 - Unnecessary re-renders
 - Memory leaks
 - Sluggish UI in large-scale apps
 - Developers must be careful with performance tuning (e.g., memoization, lazy loading, etc.).
-

♦ 6. SEO Limitations (Without Server-Side Rendering)

- React apps are **client-side rendered by default**, meaning content is generated in the browser.
 - Search engines may have trouble indexing these pages correctly.
 - **Solution:** Use SSR frameworks like **Next.js**, but that adds more complexity.
-

♦ 7. Steep Learning Curve for Advanced Features

- Concepts like **Hooks**, **Context API**, and **Concurrent Mode** can be difficult for beginners.
 - Developers must also understand **JavaScript deeply** to use React effectively.
-

Summary Table

Limitation	Description
Not a full framework	Requires third-party libraries for routing, state, etc.
Ecosystem complexity	Too many choices can be confusing
JSX	Unfamiliar to beginners
UI-focused	Not ideal for sensitive systems like banking
Performance	Can lag in large apps without optimization
SEO	Needs extra setup (e.g., Next.js)
Learning curve	Advanced features require deep JS knowledge

HTTP Status Code Categories

1. Informational Responses (100–199)

- o These indicate that the request was received and is continuing to be processed.
- o Example: 100 Continue

2. Successful Responses (200–299)

- o These indicate that the request was successfully received, understood, and accepted.
- o Example: 200 OK, 201 Created

3. Redirects (300–399)

- o These indicate that further action needs to be taken by the user agent to fulfill the request.
- o Example: 301 Moved Permanently, 302 Found

4. Client Errors (400–499)

- o These indicate that the request contains bad syntax or cannot be fulfilled.
- o Example: 400 Bad Request, 404 Not Found

5. Server Errors (500–599)

- o These indicate that the server failed to fulfill a valid request.
- o Example: 500 Internal Server Error, 503 Service Unavailable

Axios – HTTP Client Library for JavaScript

♦ What is Axios?

Axios is a **promise-based HTTP client** for the browser and Node.js that is used to make HTTP requests in a simpler and more powerful way than the native `fetch()` API.

Installation

To install Axios via npm, use:

```
npm install axios
```

Then import it into your code:

```
import axios from "axios";
```

Why Use Axios Over Fetch?

Feature	Axios	Fetch
Default JSON transform	✓ Yes	✗ No (manual needed)
Interceptors	✓ Yes (e.g., add auth headers)	✗ No
Request/response transform	✓ Yes	✗ No
Timeout support	✓ Yes	✗ No
Cancel requests	✓ Yes (via <code>CancelToken</code>)	✗ No
Wide browser support	✓ Excellent	⚠ Requires polyfills for older browsers

Features of Axios

1. **Make XMLHttpRequests from the browser**
 - o Works in all modern browsers.
2. **Make HTTP requests from Node.js**
 - o Ideal for both frontend and backend (server-side).
3. **Supports the Promise API**

- o Use `.then()`, `.catch()`, and `async/await` easily.
 - 4. **Intercept requests and responses**
 - o Modify or log requests/responses before they are handled.
 - o Example: Add auth tokens globally.
 - 5. **Transform request and response data**
 - o You can modify request data before it is sent or response data before it is handled.
 - 6. **Cancel requests**
 - o Useful in cases like fast-typing search bars or aborting API calls on component unmount.
 - 7. **Automatic JSON data transformation**
 - o Automatically converts response to JSON.
 - o Also sends JSON without needing to stringify manually.
-

Basic Axios Usage

```
axios.get('https://api.example.com/users')
  .then(response => {
    console.log(response.data); // Access response data
  })
  .catch(error => {
    console.error(error);
  });
```

Fetching Data Using Fetch API in React (with Cards)

What is fetch()?

The `fetch()` method is a built-in JavaScript API used to make HTTP requests to servers and load data asynchronously.

◆ Example: Fetching and Displaying User Data in Cards (Class Component)

```
// Importing Component from React to create a class-based component
import { Component } from "react";

// Creating a class component named 'Userdata'
class Userdata extends Component {
```

```

constructor() {
  super(); // Call the parent class (Component) constructor
  // Initializing component state
  this.state = {
    userscollection: [] // An empty array to hold user data once fetched
  };
}

// Arrow function to fetch user data from the API
fetchdata = () => {
  // Making HTTP GET request using fetch
  fetch("https://dummyjson.com/users")
    .then(response => response.json()) // Parsing response to JSON
    .then(data => {
      // Updating the component state with the fetched users
      this.setState({
        userscollection: data.users
      });
    });
};

// The render method returns JSX to be displayed in the UI
render() {
  return (
    <>
      {/* Heading */}
      <h1>User Collection</h1>

      {/* Button to trigger the fetchdata method */}
      <button onClick={this.fetchdata}>Click to Fetch User Data</button>

      {/* Mapping over the userscollection array to display user info */}
      {this.state.userscollection.map(eachobj => {
        return (
          <div
            key={eachobj.id} // Key helps React uniquely identify each element
            style={{
              border: "1px solid gray", // Adding border to each card
              borderRadius: "10px", // Rounded corners
              padding: "10px", // Inner spacing
              margin: "10px", // Spacing between cards
              width: "300px", // Fixed width for each card
              backgroundColor: "#f9f9f9" // Light background color
            }}
          >
            {/* Displaying full name by combining first and last names */}
            <h3>Full Name: {`${eachobj.firstName} ${eachobj.lastName}`}</h3>

            {/* Displaying age */}
            <p>Age: {eachobj.age}</p>
          </div>
        );
      })}
    </>
  );
}

```

```

    );
  }}
</>
);
}
}

// Exporting the component so it can be used in other parts of the app
export default Userdata;

```

Key Concepts in the Example

Concept	Explanation
fetch()	Makes an HTTP GET request to the given API URL.
.then(response => response.json())	Converts raw response to JSON format.
this.setState()	Updates the state with the fetched user data.
map()	Loops over each user and displays them in a card-like div.
key in div	Helps React identify each item uniquely in the DOM.
Inline CSS in style	Used to design simple card UI (border, padding, etc.).

Fetching Data Using axios in React (with Cards)

```

import { Component } from "react";
import axios from "axios";

class Userdata extends Component {
  state = { userscollection: [] };

  fetchdata = async () => {
    try {
      const { data } = await axios.get("https://dummyjson.com/users");
      this.setState({ userscollection: data.users });
    } catch (error) {
      alert("Something went wrong");
    }
  };

  render() {

```

```

return (
  <>
    <h1>User Collection</h1>
    <button onClick={this.fetchdata}>Click to Fetch User Data</button>

    {this.state.userscollection.map(user => (
      <div key={user.id}>
        <h3>Fullname: {`${user.firstName} ${user.lastName}`}</h3>
        <p>Age: {user.age}</p>
        <h3>-----</h3>
      </div>
    ))}
  </>
);
}
}

export default Userdata;

```

React Lifecycle Overview

Just like life has stages (birth → survival → expiry), **React components** also go through a **lifecycle**:

- **Mounting** (Birth): Component is being created and inserted into the DOM.
- **Updating** (Survival): Component is re-rendered due to state or prop changes.
- **Unmounting** (Expiry): Component is removed from the DOM.

DOM vs React Component Lifecycle

- **DOM Lifecycle**: Native browser rendering and behavior.
 - **React Component Lifecycle**: Managed by React – how components are created, updated, and destroyed.
-

Class-Based Component Lifecycle Phases

1. Mounting Phase – *Component is inserted into the DOM*

Think of this as the component being born.

a. constructor()

- **Purpose:** Initialize state and bind event handlers.
- **Auto-invoked** when the component is created.
- You must call `super(props)` to access `this.props`.

```
constructor(props) {  
  super(props);  
  this.state = { count: 0 };  
}
```

b. render()

- **Purpose:** Returns JSX to render UI.
- Pure function – no side effects here.
- Called every time the component renders (including updates).

```
render() {  
  return <h1>Hello, world!</h1>;  
}
```

c. componentDidMount()

- **Called once**, right after the component is added to the DOM.
- **Best place for side effects** like:
 - HTTP requests (API calls)
 - DOM manipulations
 - Setting up event listeners
 - Starting timers

```
componentDidMount() {  
  fetchData();  
  window.addEventListener('resize', this.handleResize);  
}
```

d. static getDerivedStateFromProps(props, state)

```
static getDerivedStateFromProps(props, state) {  
  if (props.value !== state.prevValue) {  
    return { prevValue: props.value };  
  }  
  return null;  
}
```



```
}
```

static getDerivedStateFromProps(props, state)

♦ **What is it?**

- A special **static** lifecycle method in React.
- Used **rarely** to **update the component's state** based on **changes in props**.
- **Called before every render**, during:
 - Initial mounting
 - Updates (when component receives new props)

♦ **Key Points:**

- Must be defined as a **static** method inside the class.
- It **does NOT** have access to this (because it's static).
- Should return:
 - An **object** to **update state**.
 - Or **null** if no state update is needed.

♦ **Syntax:**

```
static getDerivedStateFromProps(props, state) {  
  // logic to compare props and state  
  return updatedStateObjectOrNull;  
}
```

♦ **Parameters:**

Parameter	Description
props	The current props.
state	The current state.

♦ **When to use it?**

- When **state needs to be updated** because **props have changed**.
- Example use cases:
 - Resetting internal state when props change.
 - Keeping some state value synced with a prop.

✓ **Best practice:** Only use if absolutely necessary. Otherwise, React recommends managing data via props/state naturally without forceful syncing.

Example 1: Simple Update from Props

```
class ColorComponent extends React.Component {
  constructor(props) {
    super(props);
    this.state = { color: 'red' };
  }

  static getDerivedStateFromProps(props, state) {
    console.log("getDerivedStateFromProps called");

    if (props.favColor !== state.color) {
      return { color: props.favColor };
    }
    return null;
  }

  render() {
    return <h1>My favorite color is {this.state.color}</h1>;
  }
}
```

How it works:

- If favColor prop changes, the color state gets updated.
 - Otherwise, state remains the same.
-

React Class Component Mounting: Execution Flow

What happens when the component mounts?

When <Mountcomp /> is rendered for the first time, React goes through the **Mounting Phase**, in the following order:

1. constructor() – First Priority

- Initializes state and sets up the component.

- Called **before** anything is rendered.
- `super()` is required to use `this` in the constructor.

```
constructor() {
  console.log("first prior constructor");
  super();
  this.state = {
    message: "hello moto",
    userdata: []
  };
}
```

 **Logs:**

first prior constructor

2. `static getDerivedStateFromProps(props, state)` – Second Priority (Optional)

- Static method, **rarely used**.
- Runs **after constructor** and **before each render**, both during **mounting** and **updating**.
- Used to **update state based on prop changes**.

```
static getDerivedStateFromProps(props, state) {
  console.log("second priority - getDerivedStateFromProps");
  return null; // or return updated state
}
```

 **Logs:**

second priority - getDerivedStateFromProps

3. `render()` –third Priority

- Returns JSX to render the UI.
- React will use the return value to display the UI.
- Called **after constructor** and **every time the component updates**.

```
render() {
  console.log(this.state.userdata);
  console.log("third priority - render method");
  ...
}
```

Logs:

[] (initial empty userdata)

second priority - render method

3. componentDidMount() –fourth Priority

- Called **once only**, right after the component is mounted and the initial render is complete.
- Used for **side effects**, like:
 - API calls
 - DOM manipulation
 - Event listeners

```
componentDidMount() {  
  console.log("component did mount");  
  axios.get("https://dummyjson.com/users")  
    .then(response => {  
      this.setState({  
        userdata: response.data.users  
      });  
    });  
}
```

Logs:

component did mount

 Once data is fetched and setState is called, the component **re-renders**, which again invokes render().

Conditional Rendering

In render(), you're conditionally displaying:

- List of user names if userdata.length > 0
- <Spincomp /> loader if data is still being fetched

```
return (  
  <div>  
    {  
      this.state.userdata.length > 0 ?  
        this.state.userdata.map(eachobj => (  
          <h1 key={eachobj.id}>{eachobj.firstName}</h1>  
        )  
      )  
    }  
  )  
)
```

```

    ))
    :
    <h1><Spincomp /></h1>
  }
</div>
);

```

🔑 Mounting Phase Summary: Priority Order

Priority	Lifecycle Method	Purpose
🏆 1	constructor()	Initialize state and props
🥈 2	render()	Returns JSX to the DOM
🥉 3	componentDidMount()	Perform side effects (e.g., API calls)

Full program

```

import axios from "axios";
import { Component } from "react";
import Spincomp from "./spin";

class Mountcomp extends Component {
  constructor() {
    console.log("first prior constructor")
    super()
    this.state = {
      message: "hello moto",
      userdata: []
    }
  }

  componentDidMount() {
    console.log("component did mount")
    axios.get("https://dummyjson.com/users")
      .then(response => {
        this.setState({
          userdata: response.data.users
        })
      })
  }
}

```

```

    )
  }

  render() {
    console.log(this.state.userdata)
    console.log("second priority - render method")
    return (
      <div>
        {
          this.state.userdata.length > 0 ?
            this.state.userdata.map(eachobj => {
              return (
                <h1 key={eachobj.id}>{eachobj.firstName}</h1>
              )
            })
            :
            <h1>
              <Spincomp />
            </h1>
        }
      </div>
    )
  }
}

export default Mountcomp

```

React spinner library

react-loader-spinner Library

What is it?

react-loader-spinner is a React library that provides **beautiful loading indicators (spinners)** for your app.

- Easy to use
- Supports various spinner styles (like Audio, Bars, Oval, TailSpin, etc.)
- Can be customized with size, color, radius, and more

Installation

```
npm install react-loader-spinner --save
```

⚠ If you're using React 19 and see a peer dependency warning, use:

```
npm install react-loader-spinner --legacy-peer-deps
```

🧩 Basic Usage Example

Spin.js

```
import { Component } from "react";
import { MutatingDots } from 'react-loader-spinner'

class Spincomp extends Component {
  render() {
    return (
      <MutatingDots
        visible={true}
        height="100"
        width="100"
        color="#4fa94d"
        secondaryColor="#4fa94d"
        radius="12.5"
        ariaLabel="mutating-dots-loading"
        wrapperStyle={}
        wrapperClass=""
      />
    )
  }
}

export default Spincomp
```

🔧 Props used:

Prop	Description
height	Height of the spinner
width	Width of the spinner
radius	Corner radius

Prop	Description
color	Color of the loader
ariaLabel	Accessibility label
wrapperStyle	(optional) Inline styles for the wrapper
wrapperClass	(optional) CSS class for the wrapper

2) React Component Updating Phase

🌟 Overview of Updating Phase

The **updating phase** happens when a component's **state** or **props change**. React **re-renders** the component to reflect the new data.

The updating phase has **special lifecycle methods** that run in a specific sequence.

⚙️ Lifecycle methods during Updating Phase:

1. `getDerivedStateFromProps(props, state)`
2. `shouldComponentUpdate(nextProps, nextState)`
3. `render()`
4. `getSnapshotBeforeUpdate(prevProps, prevState)`
5. `componentDidUpdate(prevProps, prevState, snapshot)`

🔥 Your Code Breakdown

1. `getDerivedStateFromProps(props, state)`

```
static getDerivedStateFromProps(props, state) {  
  console.log("get derived state from props");  
  return { color: props.favColor };  
}
```

-  **When it is called?**
Before **render** during **mounting** and **updating**.
-  **Purpose?**
Update the internal state **based on changed props**.

- **Example in your code:**
If the parent sends a new prop favColor, this method will update the color in the state.
 - ⚡ **Note:**
It is a **static method**, so you cannot use this.
-

2. shouldComponentUpdate(nextProps, nextState)

```
shouldComponentUpdate() {  
  console.log("should update");  
  return true;  
}
```

- **When it is called?**
After getDerivedStateFromProps and **before** render().
 - **Purpose?**
Decide whether the component should **re-render or not**.
 - **Example in your code:**
Always returns true, so **always re-renders**.
(You can use this for **performance optimization** by preventing unnecessary re-renders.)
-

3. render()

```
render() {  
  console.log("render");  
  return (  
    <>  
      <h1 style={{ color: this.state.color }}>counter example  
{this.state.count}</h1>  
      <button onClick={this.increment}>+</button>  
      <button onClick={this.decrement}>-</button>  
    </>  
  );  
}
```

- **When it is called?**
After shouldComponentUpdate, before getSnapshotBeforeUpdate.
- **Purpose?**
Return **JSX** to **update the UI**.

-  **Example in your code:**
Displays the current count and two buttons to increment and decrement.
 -  **Note:**
render() must be **pure** — **no side effects** inside render (like API calls, timers, etc.)
-

4. getSnapshotBeforeUpdate(prevProps, prevState)

```
getSnapshotBeforeUpdate(prevProp, prevState) {  
  console.log(prevState, "previous state");  
  return null;  
}
```

-  **When it is called?**
Right after render() but before DOM updates.
 -  **Purpose?**
Capture some **information from the DOM** (like scroll position) **before it is changed**.
 -  **Example in your code:**
Just logging the previous state. (You can return any value you want to pass into componentDidUpdate.)
-

5. componentDidUpdate(prevProps, prevState, snapshot)

```
componentDidUpdate() {  
  console.log("com did update");  
  document.title = `component current count ${this.state.count}`;  
}
```

-  **When it is called?**
After the component has been **updated and re-rendered to the DOM**.
 -  **Purpose?**
Perform side-effects — like **API calls, DOM updates, setting document title**, etc.
 -  **Example in your code:**
After every increment/decrement, the page title updates to show the **current count**.
-

Your Example - Full Flow

User clicks + button ➡ increment() is called ➡ setState changes count ➡ Updating Phase starts:

```
import { Component } from "react";  
  
class Updatingphase extends Component{
```

```
state={
  count:0,
  color:"red"
}

increment={()=>{
  this.setState({
    count:this.state.count+1
  })
}}

componentDidMount(){
  console.log("component did mount")
  document.title=`component current count ${this.state.count} `
}

shouldComponentUpdate(){
  console.log("should update")
  // use in optimization purpose, in pure component
  return true
}

componentDidUpdate(){
  console.log("com did update")
  // api calls are also we can
  document.title=`component current count ${this.state.count} `
}

getSnapshotBeforeUpdate(prevProp,prevState){
  console.log(prevState,"previous state")
  return null
}

static getDerivedStateFromProps(props,state){
  console.log("get derived state from props")

  return {color:props.favColor}
}
```

```

render(){
  console.log("render")
  return(
    <>
      <h1 style={{color:this.state.color}}>counter example
{this.state.count}</h1>
      <button onClick={this.increment}>+</button>
    </>
  )
}
}

export default Updatingphase

```

Step	Lifecycle Method	Action
1	getDerivedStateFromProps	Adjusts color if props changed
2	shouldComponentUpdate	Checks if re-render should happen (returns true)
3	render()	Prepares the new UI with updated count
4	getSnapshotBeforeUpdate	Captures old state if needed
5	componentDidUpdate	Updates document title with new count

Key Points to Remember

- getDerivedStateFromProps is used to **sync props to state**.
- shouldComponentUpdate can **optimize performance**.
- render should be **pure** and return **JSX**.
- getSnapshotBeforeUpdate captures **info before DOM update**.
- componentDidUpdate is for **side-effects after update**.

3. Unmounting Phase – *Component is being removed from the DOM*

🌟 Overview of Unmounting Phase

- The **unmounting phase** happens when a **component is being removed** from the **DOM**.
- React provides a special **lifecycle method** for cleanup tasks.

⚙ Lifecycle method during Unmounting Phase:

Method Name	Purpose
componentWillUnmount()	Used to clean up before the component disappears from the screen

🔥 Your Code Breakdown

1. ParentComp Component

```
class ParentComp extends Component {
  constructor() {
    super();
    this.state = { istrue: true };
  }

  togglecomp = () => {
    this.setState({ istrue: !this.state.istrue });
  }

  render() {
    return (
      <>
        <button onClick={this.togglecomp}>click to change</button>
        {
          this.state.istrue ?
            <Child /> :
            <h1>child is invisible</h1>
        }
      </>
    );
  }
}

export default ParentComp;
```

- **What happens here?**

- There is a **button**.
- On clicking the button, the `isttrue` value **toggles** between `true` and `false`.
- If `isttrue` is `true`, the Child component is shown.
- If `isttrue` is `false`, the Child component **unmounts** and a message "child is invisible" is shown.

2. Child Component

```
class Child extends Component {
  componentWillUnmount() {
    alert("comp child is removed");
    console.log("we can use to clear the memory leaks");
  }

  render() {
    return (
      <h1>child is visible</h1>
    );
  }
}
```

- **componentWillUnmount():**

- **When it is called?**
Just **before** the component is **removed** from the **DOM**.
- **Purpose?**
 - Cleanup tasks:
 - Clearing timers (`setTimeout`, `setInterval`)
 - Removing event listeners
 - Canceling network requests
 - Clearing memory leaks
- **Example in your code:**
 - It **alerts** the user: "comp child is removed".
 - Logs a message: "we can use to clear the memory leaks" to the console.

Full Flow

User clicks the button → `togglecomp()` is triggered → state `isttrue` becomes false → Child component unmounts → `componentWillUnmount()` executes:

Step	Action
1	User clicks on "click to change"
2	<code>isttrue</code> becomes false
3	React decides to remove <code><Child /></code> from DOM
4	<code>componentWillUnmount()</code> in Child runs
5	Alert pops up saying "comp child is removed"
6	Console logs "we can use to clear the memory leaks"

Key Points to Remember

- `componentWillUnmount()` is called **only once, right before** the component is **destroyed**.
- It is mostly used for:
 - **Clearing timers.**
 - **Removing event listeners.**
 - **Canceling network requests.**
 - **Preventing memory leaks.**
- No `setState()` should be called inside `componentWillUnmount()`, because the component is going to be removed.

Class Component Example

```
class Timer extends React.Component {
  constructor(props) {
    super(props);
    this.state = { count: 0 };
  }

  componentDidMount() {
    this.intervalId = setInterval(() => {
      this.setState(prev => ({ count: prev.count + 1 }));
    }, 1000);
  }
}
```

```

}

componentWillUnmount() {
  clearInterval(this.intervalId); // 🛑 Prevent memory leak
}

render() {
  return <h2>Count: {this.state.count}</h2>;
}
}

```

📌 Summary

Phase	Method	Purpose
Unmounting	<code>componentWillUnmount()</code>	Cleanup tasks before the component is removed from the DOM

Here you go! 📖 Here's a **well-structured layman-friendly note** on React **Lifecycle Methods** using the **Restaurant Analogy**, perfect for beginners and teaching others.

📖 React Lifecycle Methods Explained with a Restaurant Visit Analogy

🚩 1. Mounting Phase

(The customer enters and gets settled.)

This is when the component is **created and added to the page**.

Lifecycle Method	Restaurant Analogy
constructor()	Customer enters the restaurant and shares their preferences (e.g., seating, allergies). Initial setup.
getDerivedStateFromProps()	The waiter checks if the customer has a special request in advance (like a birthday or dietary note).
render()	The waiter sets the table, places the menu — visible setup is done.
componentDidMount()	The customer is seated and ready. The waiter takes the actual food order. Now backend stuff can start (fetching food/API).

2. Updating Phase

(Customer changes something after being seated.)

This is when the component's **props or state changes** — like reordering food or changing seats.

Lifecycle Method	Restaurant Analogy
getDerivedStateFromProps()	The customer asks for an updated dish or changes their request (new props are synced).
shouldComponentUpdate()	Waiter thinks: “Should I go back to the kitchen or is this unnecessary?” (Optimize effort)
render()	Waiter brings updated table setup based on new requests (re-renders UI).
getSnapshotBeforeUpdate()	Waiter takes a quick photo of the table before replacing dishes (save scroll position, DOM info).
componentDidUpdate()	Updated dishes are served, and the waiter checks in with the customer (do something after update).

3. Unmounting Phase

(Customer finishes and leaves.)

This is when the component is **removed from the page**.

Lifecycle Method	Restaurant Analogy
componentWillUnmount()	The customer leaves. Waiter clears the table, resets it for the next guest. (Cleanup like clearing timers or unsubscribing events)

4. Error Handling Phase

(Something goes wrong — food spills or wrong dish served.)

This is when something goes wrong during rendering or logic.

Lifecycle Method	Restaurant Analogy
getDerivedStateFromError()	The manager notices the issue and decides how to handle it (update state to show fallback UI).
componentDidCatch()	Manager apologizes, fixes the error, logs it for training or reports it to the kitchen.

Final Thought

Just like a **restaurant visit**, a React component has a **start**, **middle**, and **end**. Lifecycle methods are like **waiters and staff**, managing everything from the moment you walk in until you leave — making sure everything is smooth, even if something goes wrong!

◆ React PureComponent

✓ What is PureComponent?

PureComponent is a special type of component in React that **implements** **shouldComponentUpdate()** with a **shallow prop and state comparison**.

- It helps in **preventing unnecessary re-renders** if the props and state have not changed.
- PureComponent is used **only with class components**.
- It's useful for **performance optimization**, especially in large applications.

◆ Difference Between Component and PureComponent

Feature	Component	PureComponent
Default <code>shouldComponentUpdate</code>	Always returns true	Shallowly compares props and state
Re-render behavior	Always re-renders on state/prop update	Only re-renders if shallow comparison shows changes
Use case	General-purpose component	Performance-optimized static child components

🔧 Example Explanation: Your Code

```
import { Component, PureComponent } from "react";

class Parentco extends Component {
  constructor(){
    super()
    this.state = { counter: 0 }
  }
}
```

```

inc = () => {
  this.setState({ counter: this.state.counter + 1 });
}

render(){
  console.log("parent");
  return (
    <>
      <h1>hello</h1>
      <h2>{this.state.counter}</h2>
      <button onClick={this.inc}>+</button>

      { /* <Childco counter={this.state.counter}/> */ }
    </>
  );
}
}

```

In this component:

- Parentco is a regular Component.
 - On clicking the button, the state counter increases, so Parentco re-renders.
 - The console will log "parent" each time the button is clicked.
-

```

class Childco extends PureComponent {
  render() {
    console.log("child");
    return (
      <>
        <h1>I am child</h1>
      </>
    );
  }
}

```

- Childco is a PureComponent.
 - If the parent re-renders **but the props passed to Childco do not change**, then Childco **will not re-render**.
-

⚠ Key Point in Your Code:

```
{/* <Childco counter={this.state.counter}/> */}
```

- This line is commented.
 - If **you uncomment it**, then:
 - Childco receives counter as a prop.
 - If counter **changes**, Childco **will re-render** (because PureComponent detects change in props).
 - If counter does **not** change, it won't re-render — thanks to shallow comparison.
-

✅ When to Use PureComponent?

- You have components with **same props/state across multiple renders**.
 - You want to **avoid unnecessary rendering** to improve performance.
 - You're dealing with **static data** or slow-rendering child components.
-

⚠ Notes and Limitations

1. **Shallow comparison only:** It won't detect nested object changes.
 2. `this.setState({ user: { name: "Teja" } });` // This creates a new object — triggers re-render
 3. `this.state.user.name = "Teja";` // This mutates existing object — may not trigger re-render
 4. **Functional components with `React.memo()`** behave similarly to PureComponent.
-

🔍 Summary

- PureComponent is great for performance when used **wisely**.
 - Avoid it if:
 - You rely on **deep object comparisons**.
 - Your component is already lightweight or doesn't re-render frequently.
-

Circles task

//  Circlecomp: A component to dynamically add, toggle, and delete circular buttons

```
class Circlecomp extends Component {
  constructor() {
    super()
    this.state = {
      circles: [] //  Holds boolean values: false = white, true = red
    }
  }

  //  Adds a new white circle (false)
  addcircle = () => {
    let newcrc = [...this.state.circles, false]; // Cloning and adding a new circle
    this.setState({
      circles: newcrc
    });
  }

  //  Toggles circle color at index 'ind'
  colorchange = (ind, val) => {
    //  Direct mutation of state (not recommended)
    this.state.circles[ind] = !val;
    let new1 = this.state.circles;
    console.log(ind, new1);
    this.setState({
      circles: new1
    });

    //  Recommended safer way (use this instead):
    // let new1 = [...this.state.circles];
    // new1[ind] = !val;
    // this.setState({ circles: new1 });
  }

  //  Deletes circle at index 'ind'
  delitem = (ind) => {
    let new2 = this.state.circles;
    new2.splice(ind, 1); // Removes one item at index
    this.setState({
      circles: new2
    });
    console.log(ind, new2);
  }

  render() {
    return (
      <>
        <h1>circl comp</h1>
         Add Circle Button 
        <button onClick={this.addcircle}>click to add</button>

        <div>
          {
            //  If there are circles, map and render each
            this.state.circles.length > 0 ?
              this.state.circles.map((val, ind) => (
                <div key={ind}>
                   Circle Button - toggles color 
                  <button
                    style={{
                      width: "200px",

```

```

        height: "200px",
        border: "1px solid red",
        borderRadius: "50%", // Makes it circular
        background: val ? "red" : "white" // red = true, white = false
      }}
      onClick={() => this.colorchange(ind, val)}
    >
  </button>

  { /* — Delete Button for the circle */
    <button style={{ width: "20px" }} onClick={() => this.delitem(ind)}> -</button>
  }
</div>
))
:
// 🟡 No circles available
<h1>no circles</h1>
}
</>
)
}
}

export default Circlecomp;

```

vs Class-Based vs Functional Components in React

Feature	Class-Based Component	Functional Component
Definition	Must extend from React.Component or Component	Plain JavaScript function
Syntax	Uses class syntax	Uses function syntax (ES6+)
Inheritance	extends Component required	No inheritance needed
Render Method	Must use render() to return JSX	Directly returns JSX (no render method)
State Handling	Has built-in support for state (this.state)	Initially stateless, but can use useState hook for state
Lifecycle Methods	Uses methods like componentDidMount, shouldComponentUpdate, etc.	No lifecycle methods, but similar behavior via useEffect and other hooks
Component Logic	Considered stateful	Originally stateless , but now can be stateful using hooks
Reusability	Reusability through class inheritance is limited	Hooks allow composition and reuse of logic
Code Structure	Typically more verbose and structured around lifecycle phases	More concise and easy to read (top-to-bottom flow)

Feature	Class-Based Component	Functional Component
Performance	Slightly heavier (older versions)	More lightweight and performant in modern React
Modern Usage	Less preferred in modern development	Preferred — especially with Hooks support

Hooks in Functional Components

> What are Hooks?

Hooks are built-in functions introduced in React 16.8 that allow functional components to **manage state, perform side effects, access context**, and interact with the **React lifecycle** without using class components. They provide a more concise and modular way to reuse logic across components by encapsulating functionality into custom hooks.

♦ Rules of Hooks

1.  Hooks **must only be used in functional components**.
2.  Hooks must be **called at the top level** of the component.
3.  Do **not** use hooks inside loops, conditions, or nested functions.
4.  Hooks must be **imported from React** (import { useState } from "react").

♦ Types of Hooks

1. Predefined (Built-in) Hooks

Hook	Description
useState	Adds state to functional components
useEffect	Performs side-effects (e.g., API calls, DOM updates)
useRef	Access DOM elements or persist values across renders
useContext	Access values from context API
useReducer	Manage complex state logic (like Redux)
useMemo	Memoize expensive calculations
useCallback	Memoize functions to prevent unnecessary re-renders

2. Custom (User-defined) Hooks

You can build your own hook by combining one or more built-in hooks:

```
function useCounter(initialValue = 0) {  
  const [count, setCount] = useState(initialValue);  
  
  const increment = () => setCount(prev => prev + 1);  
  const decrement = () => setCount(prev => prev - 1);  
  const reset = () => setCount(initialValue);  
  
  return { count, increment, decrement, reset };  
}
```

♦ Detailed Explanation of useState Hook

➤ Purpose:

To add and manage state in functional components.

➤ Syntax:

```
const [state, setState] = useState(initialValue);
```

- state: current state value.
- setState: function to update state.
- initialValue: the starting value of the state (can be number, string, object, array, etc.).

➤ How useState Works Internally (Destructuring Example)

React's useState returns an array with two elements. You can use **array destructuring** to assign them.

```
const numbers = [1, 2];  
const [a, b] = numbers;  
  
// a = 1  
// b = 2
```

Similarly, when using useState:

```
const [currentState, updaterFunction] = useState(0);  
  
// currentState = 0 (initial state)  
// updaterFunction = function to update the state
```


This helps in writing clean and readable code.

♦ useState Examples

✔ Example 1: Counter with switch-case

```
import { useState } from "react";

const Usestatecomp = () => {
  const [count, setCount] = useState(0);

  const handleCount = (val) => {
    switch (val) {
      case "increment":
        setCount(count + 1);
        break;
      case "decrement":
        setCount(count - 1);
        break;
      case "reset":
        setCount(0);
        break;
      default:
        break;
    }
  };

  return (
    <>
      <h1>Current counter value: {count}</h1>
      <button onClick={() => handleCount("increment")}>Increment</button>
      <button onClick={() => handleCount("decrement")}>Decrement</button>
      <button onClick={() => handleCount("reset")}>Reset</button>
    </>
  );
};

export default Usestatecomp;
```

✔ Example 2: Timer Countdown

```
import { useState } from "react";
```

```

const Timer = () => {
  const [timer, setTimer] = useState(100);

  const handleTimer = () => {
    setInterval(() => {
      setTimer((t) => t - 1); // safer with previous state
    }, 1000);
  };

  return (
    <>
      <h1>Timer value: {timer}</h1>
      <button onClick={handleTimer}>Start Timer</button>
    </>
  );
};

export default Timer;

```

✓ Example 3: Add to List

```

import { useState } from "react";

const Listadd = () => {
  const [fruits, setFruits] = useState(["apple", "banana", "mango"]);

  const addFruit = () => {
    const newFruits = [...fruits, "newfruit"];
    setFruits(newFruits);
  };

  return (
    <>
      <h1>Fruits List</h1>
      <ol>
        {fruits.map((fruit, index) => (
          <li key={index}>{fruit}</li>
        ))}
      </ol>
    </>
  );
};

```

```
<button onClick={addFruit}>Add New Fruit</button>
</>
);
};

export default Listadd;
```

Purpose of Code

This is a basic custom simulation of how `useState` works under the hood in React.

Code Breakdown

Short Notes

- `useState(initialState)` takes an initial value (0 here).
 - It sets `b = initialState`, storing the state.
 - `nn(n)` is a function that updates `b` to new value `n`.
 - Returns an array `[b, nn]` → similar to `[state, setState]`.
 - Finally, `const [a, setA] = useState(0)` destructures the returned state and updater.
-

```
function useState(initialState) {
  var b = initialState;

  function nn(n) {
    b = n; // update the state value
  }

  return [b, nn]; // return state and updater
}

const [a, setA] = useState(0);
```

✓ Summary

- useState enables functional components to maintain local state.
 - It returns a pair: the current state and a function to update it.
 - Multiple useState calls can be used for multiple state variables.
 - React re-renders the component when state is updated.
-

React Form Handling

In React, forms can be handled in two main ways:

✓ 1. Controlled Components – *React Controlled*

- **Definition:** React controls the form inputs through component state.
- **State Tracking:** Managed by React using useState.
- **Validation:** Field and form-level validations are possible.
- **Advantages:**
 - Validation logic inside component
 - Clear state tracking
 - React has full control over input elements
- **Steps:**
 0. Create a JSX form.
 1. Use useState to create local state variables for inputs.
 2. Bind state to inputs using the value attribute.
 3. Attach onChange handler to update state and handle validation.
 4. Collect values via event.target.value and validate if necessary.

Example (Controlled Component)

```
import { useState } from "react";

function ControlledComp() {
  const [username, setUsername] = useState("");
  const [usernameError, setUsernameError] = useState("");

  const usernameHandler = (event) => {
    const input = event.target.value;
```

```

setUsername(input);

if (input.length > 5) {
  setUsernameError("Username must be less than 5 characters");
} else {
  setUsernameError("");
  console.log({ name: input });
}
};

return (
  <>
    <form>
      <div className="mb-3 mt-3">
        <label htmlFor="email" className="form-label">Email:</label>
        <input
          type="text"
          className="form-control"
          id="email"
          placeholder="Enter email"
          value={username}
          onChange={usernameHandler}
        />
        {usernameError && <h1 style={{ color: "red" }}>{usernameError}</h1>}
      </div>

      <div className="mb-3">
        <label htmlFor="pwd" className="form-label">Password:</label>
        <input type="text" className="form-control" id="pwd" placeholder="Enter
password" />
      </div>

      <div className="form-check mb-3">
        <input className="form-check-input" type="checkbox" name="remember" />
        <label className="form-check-label"> Remember me </label>
      </div>

      <button type="submit" className="btn btn-primary">Submit</button>
    </form>
  </>
);
}

export default ControlledComp;

```

❌ 2. Uncontrolled Components – *DOM Controlled*

- **Definition:** The DOM manages the form inputs, not React.

- **State Tracking:** Directly handled by the browser DOM.
 - **Validation:** Not possible via React state; manual DOM checks required.
 - **Tool Used:** useRef to access input elements.
 - **When to Use:** When minimal React interaction is required or for quick data capture.
-

React useRef Hook — Complete Notes

What is useRef?

- useRef is a React **hook** that allows you to create a **mutable reference** that persists across renders.
 - It can be used to:
 - Access and interact with **DOM elements**.
 - Store **mutable values** that **do not cause re-renders** when updated.
-

useRef Syntax

```
import { useRef } from 'react';

const myRef = useRef(initialValue);
```

- initialValue can be anything: null, number, string, object, etc.
 - Returns an object: { current: initialValue }.
-

Common Use Cases of useRef

1. Accessing DOM Elements (like document.getElementById)

You can attach ref to a DOM element to access it directly.

```
function FocusInput() {
  const inputRef = useRef(null);

  const handleClick = () => {
    inputRef.current.focus(); // Directly focuses the input element
  };

  return (
    <>
```

```

    <input ref={inputRef} type="text" />
    <button onClick={handleClick}>Focus Input</button>
  </>
);
}

```

2. Storing Mutable Values Without Causing Re-renders

```

function Timer() {
  const count = useRef(0);

  const increment = () => {
    count.current += 1;
    console.log(count.current); // Logs updated value without re-rendering
    the component
  };

  return <button onClick={increment}>Increment</button>;
}

```

- Unlike useState, updating useRef().current does **not** cause the component to re-render.
- Ideal for:
 - Storing previous values
 - Keeping counters/timers
 - Skipping state-based rerenders

Key Characteristics of useRef

Feature	Description
Persistent across renders	Yes
Causes re-render on update	No
Has .current property	Yes
Can reference DOM nodes	Yes
Used in functional components	Yes

⚠ Differences Between useState and useRef

Feature	useState	useRef
Triggers re-render?	✅ Yes	❌ No
Tracks component data	✅ Yes	✅ Yes (but not for rendering)
DOM access	❌ No	✅ Yes
Syntax	const [x, setX] = useState()	const x = useRef()

📌 Example (Uncontrolled Component)

```
import { useRef } from "react";

const Uncontrolledcomp = () => {
  const usernameRef = useRef(null);
  const passwordRef = useRef(null);

  const handleForm = (event) => {
    event.preventDefault();
    const username = usernameRef.current.value;
    const password = passwordRef.current.value;

    if (username.length < 5 || password.length < 5) {
      alert("Username and password must be greater than 5 characters");
    } else {
      console.log({ name: username, mail: password });
    }
  };

  return (
    <>
      <form onSubmit={handleForm}>
        <div className="mb-3 mt-3">
          <label htmlFor="email" className="form-label">Email:</label>
          <input type="text" className="form-control" id="email"
            ref={usernameRef} placeholder="Enter email" />
        </div>
      </form>
    </>
  );
};
```



```

    <div className="mb-3">
      <label htmlFor="pwd" className="form-label">Password:</label>
      <input type="text" className="form-control" id="pwd"
ref={passwordRef} placeholder="Enter password" />
    </div>

    <div className="form-check mb-3">
      <input className="form-check-input" type="checkbox"
name="remember" />
      <label className="form-check-label"> Remember me </label>
    </div>

    <button type="submit" className="btn btn-primary">Submit</button>
  </form>
</>
);
};

export default Uncontrolledcomp;

```

Validations

1. Field-Level Validation

- Validation is applied to individual input fields.
- Example: Checking length of username or required fields.

2. Form-Level Validation

- Done when submitting the entire form.
- All fields are checked together.



useEffect in React

What is useEffect?

The useEffect hook lets you perform **side effects** in **function-based components** — operations like:

-  Data fetching (e.g., from an API)

-  DOM manipulation (like updating `document.title`)
-  Adding/removing **event listeners**
-  Setting up **timers** (`setTimeout`, `setInterval`)
-  Handling **subscriptions**

`useEffect` replaces lifecycle methods like `componentDidMount`, `componentDidUpdate`, and `componentWillUnmount` from class components.

Syntax of `useEffect`

```
useEffect(() => {  
  // side-effect code here  
  
  return () => {  
    // cleanup code here (optional)  
  };  
}, [dependencies]);
```

- **First argument:** A **callback function** to execute side effects.
 - **Second argument:** A **dependency array** that determines when the effect runs.
-

Behavior Based on Dependency Array

Case 1: No dependency array

```
useEffect(() => {  
  // runs after every render  
});
```

- Acts like **`componentDidUpdate`**
 - Executes after every re-render of the component.
-

Case 2: Empty dependency array

```
useEffect(() => {  
  // runs only once after initial render  
}, []);
```

- Acts like **`componentDidMount`**

- Runs only once when the component is first mounted.

✓ Case 3: Dependency array with values

```
useEffect(() => {  
  // runs when `counter` changes  
}, [counter]);
```

- Acts like **componentDidUpdate** (but only for specific values)
- Runs only when values in the dependency array change.

4. Cleanup function (componentWillUnmount)

```
useEffect(() => {  
  const interval = setInterval(() => {  
    console.log("Running interval");  
  }, 1000);  
  
  return () => {  
    clearInterval(interval); // Cleanup  
    console.log("Cleanup interval");  
  };  
}, []);
```

- Useful for **clearing timers, removing event listeners, closing subscriptions**, etc.

Examples

✓ Example 1: DOM Manipulation (document.title)

```
import { useEffect, useState } from "react";  
  
function Domex1() {  
  const [count, setCount] = useState(0);  
  const [count1, setCount1] = useState(0);  
  
  useEffect(() => {  
    document.title = `Current count ${count}`;  
    console.log("useEffect ran for count");  
  });  
}
```

```

}, [count]); // runs only when `count` changes

return (
  <>
    <h1>Count value: {count}</h1>
    <button onClick={() => setCount(count + 1)}>Increment</button>

    <div>-----</div>

    <h1>Count1 value: {count1}</h1>
    <button onClick={() => setCount1(count1 + 1)}>Increment 2</button>
  </>
);
}
export default Domex1;

```

✓ Example 2: Data Fetching (Fake Store API)

```

import axios from "axios";
import { useEffect, useState } from "react";
import Spincomp from "../day8/spin";

function Apiex2() {
  const [products, setProducts] = useState([]);

  useEffect(() => {
    axios.get("https://fakestoreapi.com/products/")
      .then((response) => {
        setProducts(response.data);
        console.log(response.data);
      });
  }, []); // run only once

  return (
    <>
      <div>useEffect example</div>
      {products.length > 0 ? (
        products.map((product, index) => (
          <div key={index}>
            <h2>{product.title}</h2>
            <img src={product.image} alt="" width="100px" height="100px" />
          </div>
        ))
      ) : (
        <Spincomp />
      )}
    </>
  );
}

```

```
export default Apiex2;
```

✓ Example 3: Event Listener + Cleanup

```
import { useEffect, useState } from "react";

function Eventex3() {
  const [x, setX] = useState(null);

  useEffect(() => {
    const handleMouseMove = (event) => {
      setX(event.clientX);
      console.log("Mouse X:", event.clientX, "Mouse Y:", event.clientY);
    };

    window.addEventListener("mousemove", handleMouseMove);

    // Cleanup function - acts like componentWillUnmount
    return () => {
      window.removeEventListener("mousemove", handleMouseMove);
      console.log("Unmounting: removed event listener");
    };
  }, []); // run once on mount

  return <h1>X Axis: {x}</h1>;
}

export default Eventex3;
```

🔧 Summary

Case	Dependency Array	Behavior
No array	none	Runs after every render
Empty array	[]	Runs only once on mount
With values	[val]	Runs only when val changes

Complete Notes on React Router DOM (v6+)

What is React Router?

React Router is a standard **routing library for React**. It allows developers to build **Single Page Applications (SPAs)** with **dynamic navigation** and **client-side routing**—without full page reloads.

Why React Router?

- React is not opinionated about routing.
 - By default, it renders everything in a single DOM tree.
 - Native HTML navigation (<a>, window.location.href) causes **page reloads** and breaks SPA behavior.
 - So, we use **react-router-dom** to manage routes efficiently.
-

Types of Routing

1. Static Routing

- Routes are defined explicitly.
- URLs remain the same each time.

```
<Route path="/about" element={<About />} />
```

2. Dynamic Routing

- Uses route parameters to handle dynamic content.

```
<Route path="/:category/:id" element={<Productdetails />} />
```

Installation

```
npm install react-router-dom
```

Basic Setup

Step 1: Import Core Modules

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
```

Step 2: Wrap App in BrowserRouter

```
<BrowserRouter>
  <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/about" element={<About />} />
  </Routes>
</BrowserRouter>
```

📁 Example (Static + Dynamic Routing)

File: Day14.jsx

```
import { BrowserRouter, Route, Routes } from "react-router-dom";
import About from "./pages/about";
import Contact from "./pages/contact";
import Home from "./pages/home";
import Setting from "./pages/setting";
import Productdetails from "./pages/productdetail";

function Day14() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" Component={Home} />
        <Route path="/about" Component={About} />
        <Route path="/contact" Component={Contact} />
        <Route path="/setting" Component={Setting} />
        <Route path="/:category/:id" Component={Productdetails} />
      </Routes>
    </BrowserRouter>
  );
}

export default Day14;
```

Navigation Using <Link>

File: Navbarcomp.jsx

```
import React from 'react';
import { Link } from 'react-router-dom';

function Navbarcomp() {
  return (
    <nav className="navbar navbar-expand-sm bg-light navbar-light">
      <div className="container-fluid">
        <ul className="navbar-nav">
          <li className="nav-item m-3">
            <Link to="/">Home</Link>
          </li>
          <li className="nav-item m-3">
            <Link to="/about">About</Link>
          </li>
          <li className="nav-item m-3">
            <Link to="/contact">Contact</Link>
          </li>
          <li className="nav-item m-3">
            <Link to="/setting">Setting</Link>
          </li>
        </ul>
      </div>
    </nav>
  );
}

export default Navbarcomp;
```

Dynamic Routing

1. Dynamic Route Linking:

```
<Link to={`/${product.category}/${product.id}`}>
  View more
</Link>
```

- o This creates links like /electronics/2 or /jewelery/10.

- o Perfect use of dynamic routing with URL params (:category/:id).

2. Route to handle this must be set:

```
<Route path="/:category/:id" element={<Productdetails />} />
```

- o Productdetails will extract the category and id using useParams().
3. **Wrap the entire app with <BrowserRouter>** (already done in your earlier setup).
 4. **Use Link instead of <a>:**
 - o  <Link> avoids full page reloads and keeps SPA behavior intact.

Common Components & Hooks

Feature	Purpose
BrowserRouter	Wraps the app, uses HTML5 history API
Routes	Wrapper for multiple Route components
Route	Maps a path to a component
Link	Replaces <a> for navigation (no reload)
useNavigate()	For programmatic navigation
useParams()	To read route parameters (e.g., /user/:id)
useLocation()	To read current URL, state, search params
Navigate	Redirect to another route
Outlet	Renders child routes inside nested routes

+ Additional Features

 **React Router DOM – useNavigate and useParams (v6+)**

 **useNavigate – Navigation Programmatically**

 **What is useNavigate?**

- `useNavigate` is a hook provided by `react-router-dom` used to programmatically navigate between routes.
- Replaces the older `useHistory` in React Router v5.

Syntax:

```
import { useNavigate } from 'react-router-dom';

const Component = () => {
  const navigate = useNavigate();

  const goToPage = () => {
    navigate('/path'); // navigate to specified route
  };

  return <button onClick={goToPage}>Go</button>;
};
```

Parameters of `navigate(path, options)`

Parameter	Description
<code>path</code>	Target path/URL to navigate to.
<code>options</code>	(optional) { <code>replace</code> : boolean, <code>state</code> : any }

Options Example:

```
navigate('/dashboard', { replace: true, state: { from: 'login' } });
```

- `replace: true` replaces current history entry (like `redirect`).
- `state` lets you pass additional data to the target route.

Example 1: Navigate to Home on Invalid URL

```
// Invalidscreen.js
import React from 'react';
import { useNavigate } from 'react-router-dom';

function Invalidscreen() {
  const navigate = useNavigate();
```

```
const navigatohome = () => {
  navigate("/");
};

return (
  <div>
    <h5>URL not found, 404 status</h5>
    <button onClick={navigatohome}>Click to Home</button>
  </div>
);
}

export default Invalidscreen;
```

Explanation:

If the user lands on a non-existing page, this component is shown.

On button click, it uses `useNavigate()` to redirect the user to the homepage (`"/"`).

useParams – Get Route Parameters

What is useParams?

- `useParams` is a hook used to extract dynamic URL parameters from the current route.

Syntax:

```
import { useParams } from 'react-router-dom';

const UserDetails = () => {
  const { id } = useParams(); // Assuming URL is /user/:id

  return <h1>User ID: {id}</h1>;
};
```

Typical Route Setup:

```
<Route path="/user/:id" element={<UserDetails />} />
```

Note:

- `useParams()` returns an object where keys match the route's dynamic segments.

- All values are strings (even numbers must be parsed if needed).

🔗 Example 2: Fetch Product Details Using id from URL

```
// Productdetails.js
import axios from 'axios';
import React, { useState, useEffect } from 'react';
import { useParams } from 'react-router-dom';
import Spincomp from '../day8/spin';
import './Productdetails.css';

function Productdetails() {
  const [data, setData] = useState({});
  const info = useParams(); // info.id comes from URL

  useEffect(() => {
    fetchProducts();
  }, [info.id]);

  async function fetchProducts() {
    const result = await axios.get(`https://fakestoreapi.com/products/${info.id}`);
    setData(result.data);
  }

  return (
    <div className="detail-container">
      {Object.keys(data).length > 0 ? (
        <div className="detail-card">
          <div className="detail-left">
            <img src={data.image} alt={data.title} className="detail-image" />
          </div>
          <div className="detail-right">
            <h1 className="detail-title">{data.title}</h1>
            <p className="detail-description">{data.description}</p>
            <p className="detail-price">$ {data.price}</p>
            <p className="detail-rating">★ {data.rating?.rate}
              ({data.rating?.count} reviews)</p>
          </div>
        </div>
      ) : (
        <Spincomp />
      )}
    </div>
  )
}
```

```

</div>
);
}

export default Productdetails;

```

Explanation:

- The route is expected to be like: /product/:id.
- useParams() retrieves id from the URL, which is then used to fetch product data from an API.
- Data is conditionally rendered with a loading spinner (Spincomp) while the fetch is in progress.

Common Use-Cases

Hook Use Case Example

useNavigate Redirect after login, button-based routing

useParams Fetching a user/product/post by ID from the URL

Access Query Parameters (useSearchParams)

```

const [searchParams] = useSearchParams();
console.log(searchParams.get("name"));

```

Access Current Location (useLocation)

```

const location = useLocation();
console.log(location.pathname);

```

Nested Routes

```

<Route path="/dashboard" element={<Dashboard />}>
  <Route path="profile" element={<Profile />} />
  <Route path="settings" element={<Settings />} />
</Route>

```

Use <Outlet /> in Dashboard to show nested content.

Not Found Route (Fallback)

```
<Route path="*" element={<NotFound />} />
```

Protected Routes in React Router DOM

What is a Protected Route?

A **Protected Route** (or **Private Route**) is a route that only allows access to authenticated users. If a user is **not logged in**, they are typically **redirected to a login page** or shown an **unauthorized access message**.

Concept in Your Example

Your component Day14 uses a true condition (which can later be replaced by an authentication check) to **conditionally render routes**.

When condition is true (user is authenticated):

```
<Routes>
  <Route path="/" Component={Home} />
  <Route path="/about" Component={About} />
  <Route path="/contact" Component={Contact} />
  <Route path="/setting" Component={Setting} />
  <Route path="/:category/:id" Component={Productdetails} />
  <Route path="*" Component={Invalidscreen} />
</Routes>
```

When condition is false (user is not authenticated):

```
<Routes>
  <Route path="/" Component={Login} />
</Routes>
```

How to Make This Dynamic with Auth Check

1. Replace true with an isAuthenticated flag.
2. This flag can come from:
 - o **State**
 - o **Context**
 - o **Redux**
 - o **LocalStorage/token**

Example Update with isAuthenticated:

```

const isAuthenticated = true; // Replace with actual logic

<BrowserRouter>
  {isAuthenticated ? (
    <Routes>
      {/* Protected Routes */}
      <Route path="/" Component={Home} />
      <Route path="/about" Component={About} />
      <Route path="/contact" Component={Contact} />
      <Route path="/setting" Component={Setting} />
      <Route path="/:category/:id" Component={Productdetails} />
      <Route path="*" Component={Invalidscreen} />
    </Routes>
  ) : (
    <Routes>
      {/* Public Route */}
      <Route path="/" Component={Login} />
      <Route path="*" Component={Login} />
    </Routes>
  )}
</BrowserRouter>

```

Summary Table

Concept	Syntax / Hook	Purpose
Router Wrapper	<BrowserRouter>	Wraps entire app for routing
Define Routes	<Routes>, <Route>	Maps URL paths to components
Link Navigation	<Link to="/about" />	Navigation without page reload
Dynamic Params	/product/:id + useParams()	Fetch dynamic route values
Redirect	<Navigate to="/login" />	Redirect to another route
Programmatic Nav	useNavigate()	Navigate via function call
Nested Routing	<Outlet />	Show nested route components

Concept	Syntax / Hook	Purpose
Not Found	<code><Route path="*"></code>	Catch-all route
:		



State Management in React

What is State Management?

- State management is a way to **control and manage data flow** within a React application.
 - As applications grow in size and complexity, managing data across many components becomes more difficult.
 - **State management** ensures a smooth, predictable, and efficient way to handle such data.
-

Types of State Management

1. Local State Management

- Uses hooks like `useState`, `useReducer`.
- State is maintained **within a single component**.
- Ideal for small, isolated states (e.g., input fields, toggles).

2. Global State Management

- Needed when state must be **shared across many unrelated components**.
 - Involves **centralized** state handling.
 - Two main ways in React:
 - **Context API (built-in)**
 - **Redux (external library)**
-

Props Drilling Problem

- In React, data is typically passed **from parent to child using props**.
- When deeply nested components require the same data, it must be passed through **multiple layers**, even if intermediate components don't use it.
- This is called **props drilling**.

Drawbacks:

- Harder to maintain
- More verbose code

- Unnecessary re-renders of intermediate components

✓ Solution: Context API

Context API in React

Purpose:

The Context API allows you to **share data globally** across components **without passing props manually at every level**.

Steps to Use Context API

1 Create a Context

Use the `createContext()` function to create a context object.

```
import { createContext } from 'react';

export const store = createContext();
```

2 Provide Context (with Provider)

Wrap the component tree (or part of it) using the `store.Provider` and pass the data using the `value` prop.

```
function Day16() {
  const [data, setData] = useState("john");

  const messageUpdater = () => {
    setData("jessy");
  };

  return (
    <store.Provider value={{ data, fun: messageUpdater }}>
      <Comp1 />
    </store.Provider>
  );
}
```

- `data` is the current state ("john").
 - `fun` is the function to update it (`setData("jessy")`).
 - `Comp1` and its children can access this shared data.
-

3 Consume Context with useContext

Use the useContext() hook in any child component to read or update the shared data.

```
import { useContext } from 'react';
import { store } from './day16';

function Comp2() {
  const result = useContext(store);

  return (
    <div>
      Comp2 the name is {result.data}
      <button onClick={result.fun}>click to change</button>
    </div>
  );
}
```

- result.data accesses "john" (or "jessy" after change).
- result.fun updates the name from "john" to "jessy" on button click.

Summary of Your Code Flow

- Day16: Defines and provides context (with state and function).
- Comp1: Intermediate component (not shown here) that doesn't need props.
- Comp2: Directly uses useContext() to read and modify context values.

useContext Hook Overview

Syntax:

```
const contextValue = useContext(YourContext);
```

Features:

- Simplifies accessing global data.
- Eliminates need for props drilling.
- Automatically re-renders components on context value change.

Summary Table

Feature	Description
Tool	useContext (React Hook)
Use Case	Access shared/global data across components
Benefit	Removes props drilling and simplifies data sharing
Setup Needed	createContext(), Provider, useContext()
Replaces	Manually passing props through many component levels



React useReducer Hook

What is useReducer?

useReducer is a React Hook used for **managing complex state logic** in functional components, especially when:

- State has multiple sub-values.
 - State updates depend on the previous state.
 - You want to centralize update logic (like in Redux).
-

Syntax

```
const [state, dispatch] = useReducer(reducerFunction, initialState);
```

- **state** → current state value.
 - **dispatch** → function to trigger state changes.
 - **reducerFunction** → function that takes current state and an action, and returns new state.
 - **initialState** → the initial state object/value.
-

Reducer Function Structure

```
function reducer(state, action) {  
  switch (action.type) {  
    case "actionType":
```

```

    return { ...state, key: updatedValue };
  default:
    return state;
  }
}

```

- **Pure function:** Must not modify state directly. Always return a new state object.
- **action:** An object, commonly { type: 'actionType', payload: someData }.

Example

```

import React, { useReducer } from 'react';

const initialState = { count: 0 };

function reducer(state, action) {
  switch (action.type) {
    case 'increment':
      return { count: state.count + 1 };
    case 'decrement':
      return { count: state.count - 1 };
    default:
      return state;
  }
}

function Counter() {
  const [state, dispatch] = useReducer(reducer, initialState);

  return (
    <div>
      <p>Count: {state.count}</p>
      <button onClick={() => dispatch({ type: 'increment' })}>+</button>
      <button onClick={() => dispatch({ type: 'decrement' })}>-</button>
    </div>
  );
}

export default Counter;

```

When to Use useReducer

Use Case	Recommended?
Multiple related state variables	 Yes
Complex state update logic	 Yes

Use Case	Recommended?
Managing state with many conditionals	✔ Yes
Simple toggle or counter	✗ Better to use useState
Centralizing state transitions (Redux style)	✔ Yes

📁 Action Object Structure

An action usually has:

```
{
  type: "actionType",
  payload: value // (optional)
}
```

Example with payload:

```
case "updateName":
  return { ...state, name: action.payload }
```

🔧 Advanced Example

```
import React, { useReducer } from 'react';

const initialState = {
  age: 17,
  salary: 20000,
  name: "john"
};

function reducer(state, action) {
  switch (action.type) {
    case "increment":
      return { ...state, age: state.age + action.payload };
    case "decrement":
      return { ...state, age: state.age - action.payload };
    case "incrementsal":
      return { ...state, salary: state.salary + 1000 };
    case "decrementsal":
      return { ...state, salary: state.salary - 1000 };
    case "updatename":
      return { ...state, name: action.payload };
    default:
      return state;
  }
}
```

```

}
}

function Day17() {
  const [state, dispatch] = useReducer(reducer, initialState);
  const dispatchAction = (type,payload) => dispatch({ type,payload });

  return (
    <div style={{ padding: '20px' }}>
      <h3>Age: {state.age}</h3>
      <button onClick={() => dispatchAction("increment",1)}>Increase Age</button>
      <button onClick={() => dispatchAction("decrement",1)}>Decrease Age</button>

      <h3>Salary: ₹{state.salary}</h3>
      <button onClick={() => dispatchAction("incrementsal")}>Increase Salary</button>
      <button onClick={() => dispatchAction("decrementsal")}>Decrease Salary</button>

      <h3>Name: {state.name}</h3>
      <button onClick={() => dispatchAction("updatename","jessy")}>Change Name</button>
    </div>
  );
}

export default Day17;

```

⚙️ Pros of useReducer

- ✓ Better state organization
- ✓ Easier to test (pure reducer functions)
- ✓ Centralizes update logic
- ✓ Great for large applications or when using Context API

🔴 Pitfalls

- ✗ Verbose for very simple state
- ✗ More boilerplate than useState
- ✗ Must return a valid state always (even for unknown actions)

👥 useReducer with useContext (Global State)

useReducer can be combined with useContext to manage **global app-level state**, like Redux but without external libraries.

🧠 Conceptual Flow

Component -> dispatch(action)



reducer(state, action)

↓
returns new state
↓
React re-renders component

React useMemo() Hook

What is useMemo()?

useMemo is a React Hook used to **optimize performance** by **memoizing** the result of an expensive computation so that it only recalculates when its dependencies change.

Why use useMemo()?

In React, every time a component re-renders:

- All expressions and functions inside the render body run again.
- This can become **expensive** if you have **heavy calculations** or **frequently updated components**.

useMemo helps **avoid unnecessary recalculations** by remembering the result of a function between renders.

Syntax

```
const memoizedValue = useMemo(() => {  
  // expensive calculation  
  return result;  
}, [dependency1, dependency2]);
```

- Returns the **cached value** until dependencies change.
 - Only re-runs the function **if dependencies change**.
-

Flow

1. Component renders
 2. React checks if any dependency changed
 3. If yes → run function and store result
 4. If no → return cached value
-

✓ When to Use useMemo

Use Case

Heavy/expensive calculations

Filtering, sorting, looping large data

Recalculating only when specific state/props change

Simple value or quick logic

Use useMemo?

✓ Yes

✓ Yes

✓ Yes

✗ No (not worth the overhead)

🔧 Example

```
import React, { useState, useMemo } from 'react';

function Usememocomp() {
  const [count, setCount] = useState(0);
  const [otherCount, setOtherCount] = useState(0);

  const isEven = useMemo(() => {
    console.log("Running expensive check...");
    for (let i = 0; i < 1000000000; i++) {} // simulate heavy task
    return count % 2 === 0 ? "Even" : "Odd";
  }, [count]);

  return (
    <div>
      <h2>Count: {count} ({isEven})</h2>
      <button onClick={() => setCount(count + 1)}>Increment Count</button>

      <h2>Other Count: {otherCount}</h2>
      <button onClick={() => setOtherCount(otherCount + 1)}>Increment Other Count</button>
    </div>
  );
}
```

🧠 What's happening?

- The heavy function (isEven) only runs when count changes.
- Updating otherCount doesn't re-run the expensive function — **memoized value is used**.

⚠️ Common Mistakes

1. ✗ **Calling memoized value as a function:**

```
const value = useMemo(() => compute(), []);
```

```
value(); // ✗ wrong (value is not a function)
```


2. ❌ Missing dependencies:

- o Always include all values used inside the function in the dependency array.

`useMemo(() => heavyCalc(a, b), [a, b]); // ✅ correct`

3. ❌ Overuse:

- o Don't use `useMemo` for everything. It adds complexity and might hurt performance if misused.

Real-world Use Cases

- Optimizing large list filtering/sorting
- Avoiding re-calculations on form fields
- Preventing unnecessary renders in child components (when used with `React.memo`)

useMemo vs useCallback

Hook	Memoize s	Use For
<code>useMemo</code>	Value	Expensive calculations
<code>useCallback</code>	Function	Passing stable functions to children

Summary

- `useMemo` is a **performance optimization hook**.
- Caches the **result of a function**, re-running only when dependencies change.
- Best used for **heavy calculations** or **derived data**.
- Avoid using it unnecessarily for simple values.

React.memo

What is `React.memo`?

`React.memo` is a **higher-order component** used to optimize the performance of functional components by **memoizing** them. It helps avoid **unnecessary re-renders** when the props do not change.

✓ Syntax

`const MemoizedComponent = React.memo(Component);`

Optionally, with a custom comparison function:

`const MemoizedComponent = React.memo(Component, areEqual);`

🧠 Core Concept

- `React.memo` checks if props have changed using a **shallow comparison**.
 - If **props haven't changed**, it **skips re-rendering** the component.
 - Useful for **pure components** that always render the same output given the same props.
-

🔧 Your Example with Explanation

📁 Memoparent.js

```
import React, { useState } from 'react'
import Memochild from './memochild'

function Memoparent() {
  const [count, setCount] = useState(0)

  return (
    <div>
      <div>Memoparent count is {count}</div>
      <button onClick={() => setCount(count + 1)}>inc</button>
      <Memochild />
    </div>
  )
}

export default Memoparent
```

📄 Output Behavior:

- Clicking the **"inc"** button increases count and re-renders `Memoparent`.
 - But since `Memochild` **does not receive any props**, and it's wrapped in `React.memo`, it **does not re-render**.
 - Even though you see `console.log("child re renders everytime")`, after applying `React.memo`, this will **not print** on every button click — only on the **initial render**.
-

📁 Memochild.js

```
import React from 'react'

function Memochild() {
  console.log("child re renders everytime")
  return <div>Memochild</div>
}

export default React.memo(Memochild)
```

✓ You've wrapped Memochild with React.memo, so it **won't re-render** unless its props change.

✓ What This Shows

- This example **demonstrates the effect of React.memo clearly**.
 - Without React.memo, Memochild would re-render every time Memoparent re-renders.
 - With React.memo, the **child remains stable**, improving performance when child rendering is expensive.
-

🔥 Summary: When to Use React.memo

✓ Use it when:

- The component receives props that don't often change.
- Rendering is expensive and affects performance.
- You want to prevent unnecessary renders of child components.

⚠ Avoid it when:

- Props change frequently.
 - Rendering is already cheap.
 - It adds more complexity than benefit.
-

React useCallback Hook

🔍 What is useCallback?

useCallback is a **React Hook** used to **memoize a function**, so it doesn't get recreated on every render unless its **dependencies change**.

✓ Syntax

```
const memoizedFunction = useCallback(() => {  
  // logic  
}, [dependencies]);
```

🎯 Why Use useCallback?

- In React, functions are **recreated on every render**.
- If these functions are passed to **memoized child components** (using React.memo), they can cause **unnecessary re-renders**.
- useCallback keeps the **same function reference**, avoiding that.

🔧 Your Example

📁 Callchildbtn.js

```
import React from 'react';  
  
function Callchildbtn({ children, onClick }) {  
  console.log(children); // Log to check when it re-renders  
  console.log(onClick);  
  return (  
    <div>  
      <button onClick={onClick}>{children}</button>  
    </div>  
  );  
}  
  
export default React.memo(Callchildbtn);
```

✓ Callchildbtn is memoized using React.memo, so it **only re-renders when its props change**.

📁 Callparent.js

```
import React, { useCallback, useState } from 'react';  
import Callchildbtn from './callchildbtn';  
  
function Callparent() {
```

```

const [age, setAge] = useState(0);
const [sal, setSal] = useState(1000);

// Memoize age handler
const handleage = useCallback(() => {
  setAge(age + 1);
}, [age]);

// Memoize salary handler
const handlesal = useCallback(() => {
  setSal(sal + 100);
}, [sal]);

return (
  <div>
    <div>Age is {age}</div>
    <div>Sal is {sal}</div>
    <Callchildbtn onClick={handleage}>Click to change age</Callchildbtn>
    <Callchildbtn onClick={handlesal}>Click to change sal</Callchildbtn>
  </div>
);
}

export default Callparent;

```

Explanation

- Each Callchildbtn is passed a function (handleage or handlesal) as a prop.
- If we **didn't use useCallback**, these functions would be **new references on every render**, causing re-renders of the child.
- With useCallback, the functions **stay the same** between renders unless their dependencies (age, sal) change.
- This **prevents unnecessary re-renders**, improving performance.

When to Use useCallback

Use it:

- When passing **functions as props** to React.memo components.
- When functions cause **re-renders in child components** unnecessarily.

Don't use it:

- For functions used **only inside the same component**.
- When the **cost of re-render is low** or memoization adds complexity.

Recap: useCallback vs React.memo

Hook / Feature	Purpose	Avoids Re-rendering
useCallback	Memoizes functions	Yes (function stays the same)
React.memo	Memoizes components	Yes (component skips re-render if props don't change)

◆ Custom Hooks in React

✓ What are Custom Hooks?

Custom Hooks are **JavaScript functions** in React that **start with use** and **allow you to extract and reuse logic** across multiple components.

They follow the **same rules as built-in hooks** like `useState`, `useEffect`, etc., and can **use other hooks** inside them.

✓ Why Use Custom Hooks?

- Reusability: Share logic between components.
 - Cleaner Code: Keeps components focused on rendering.
 - Separation of Concerns: Decouple stateful logic from UI.
 - Testability: Easier to test isolated logic.
-

✓ Syntax

```
import { useState } from "react";

function useCustomHook() {
  const [state, setState] = useState(0);
  const updateState = () => setState(state + 1);

  return [state, updateState];
}
```

- Always start with `use`
 - Can accept arguments
 - Can return anything: state, functions, objects, etc.
-

✓ Rules of Hooks (Apply to Custom Hooks too)

1. **Only call hooks at the top level** (not inside loops, conditions).
 2. **Only call hooks from React functions** (functional components or other hooks).
-

🔧 Your Code Examples with Explanation

✓ 1. Basic useCounter Hook

📁 useCounter.js

```
import { useState } from "react";

function useCounter(){
  const [count, setCount] = useState(0)

  const handleinc = () => {
    setCount(count + 1)
  }

  return [count, handleinc]
}

export default useCounter
```

✓ Using Basic useCounter in a Component

📁 Page.js

```
import React from 'react'
import useCounter from './useCounter'

function Page() {
  const [count, setCount] = useCounter()

  return (
    <div>
      count is {count}
      <button onClick={setCount}>click to increment</button>
    </div>
  )
}
```

```
export default Page
```

Explanation of Above Code:

- useCounter encapsulates useState and increment logic.
- Page component just uses the hook, making the code cleaner.
- Custom hook returns an array: [value, function] – similar to how useState works.
- setCount here is actually your custom function handleinc.

✓ 2. Enhanced useCounter Hook with Parameters

useCounter.js

```
import { useState } from "react";

function useCounter(initial, val){
  const [count, setCount] = useState(initial)

  const handleinc = () => {
    setCount(count + val)
  }

  return [count, handleinc]
}

export default useCounter
```

✓ Using Enhanced useCounter in a Component

Page.js

```
import React from 'react'
import useCounter from './useCounter'

function Page() {
  const [count, setCount] = useCounter(10, 3)
  const [sal, setSal] = useCounter(1000, 200)

  return (
```



```

    <div>
      count is {count}
      <button onClick={setCount}>click to increment</button>
      <br/>
      salary is {sal}
      <button onClick={setSal}>click to increase</button>
    </div>
  )
}

export default Page

```

Explanation of Above Code:

- useCounter(10, 3) → count starts at 10, increases by 3.
- useCounter(1000, 200) → salary starts at 1000, increases by 200.
- This version is **dynamic and reusable**, suitable for any numeric state logic (counter, salary, score, etc.).

Summary Table

File	Purpose	Highlights
useCounter.js (v1)	Basic counter logic	Fixed initial value = 0, step = 1
Page.js (v1)	Displays count	Uses basic hook
useCounter.js (v2)	Reusable counter logic	Accepts initial, val as parameters
Page.js (v2)	Displays count & salary	Uses hook twice with different values

Custom Hook for Data Fetching with Axios – Full Notes

Why Create a Custom Hook for API Calls?

Instead of repeating the axios.get(...) + useEffect(...) logic in every component, you can:

- Encapsulate the API call in a **reusable hook**

- Handle data fetching and state management once
- Keep components clean and focused on rendering

Your Code and Explanation

useAxios.js – Custom Hook for Axios Calls

```
import axios from "axios";
import { useEffect, useState } from "react";

function useAxios(url){
  const [res, setResult] = useState([])

  useEffect(() => {
    fetchData()
  }, [])

  async function fetchData() {
    const result = await axios.get(url)
    setResult(result.data)
  }

  return [res]
}

export default useAxios
```

Explanation

- **useState([])**: Initializes an empty array to store fetched data.
- **useEffect(...)**: Runs once when the component mounts.
- **fetchData()**: Asynchronous function to call the API using `axios.get(url)`.
- **setResult(result.data)**: Updates state with the fetched data.
- **return [res]**: Returns the data to the component using the hook.

 You can reuse this hook to fetch data from any API just by passing a new url.

Page1.js – Using the useAxios Hook

```

import React from 'react'
import useAxios from './useaxios'

function Page1() {
  const url = "https://fakestoreapi.com/products/"
  const [products] = useAxios(url)

  console.log(products)

  return (
    <div>
      {
        products.map((val, id) => {
          return (
            <div key={val.id}>{val.title}</div>
          )
        })
      }
    </div>
  )
}

export default Page1

```

Explanation

- Imports the useAxios hook.
- Passes the API endpoint `https://fakestoreapi.com/products/` as an argument.
- Receives products (the data) from the hook.
- Renders the product titles using `.map()`.

✓ Simple and clean! No `useEffect` or `axios` logic in the component — it's handled by the hook.

Summary Table

File	Purpose	Key Feature
<code>useAxios.js</code>	Custom hook to fetch API data	Encapsulates Axios + <code>useEffect</code>

File	Purpose	Key Feature
Page1.js	Component that shows product titles	Uses the hook and displays data

Best Practices (for Data Fetching Hooks)

- Return error and loading state too if needed:
 - `return { data, loading, error }`
 - Use dependency array in `useEffect` for dynamic URLs:
 - `useEffect(() => {`
 - `fetchData()`
 - `}, [url])`
 - Add error handling (try-catch) in async functions.
-

Best Practices for Custom Hooks

1. Name them starting with `use` (`useCounter`, `useAuth`, `useFetch`, etc.).
 2. Return only what's necessary — avoid exposing too much.
 3. Use Prop Drilling or Context if you need to pass hook values deeply.
 4. Keep hooks focused — avoid mixing unrelated logic.
 5. Create a `hooks/` folder to organize them in large projects.
-

Summary

Concept	Example from You
Simple Custom Hook	<code>useCounter()</code>
Reusability	Used in Page multiple times
Customization	Parameters: <code>initial</code> , <code>step</code>
Clean UI Logic	Page just handles display and events

Redux – Complete Notes with Examples

1. What is Redux?

Redux is a predictable state management library for JavaScript apps, especially React. It helps manage and centralize the state of your application in one global store.

✓ Key Concepts:

- **Store** – The global state container.
 - **Actions** – Plain JavaScript objects describing "what happened".
 - **Reducers** – Functions that determine how the state changes based on the action.
 - **Dispatch** – A method used to send actions to reducers.
 - **Selector** – Used in React to get data from the store.
-

◆ 2. Why Do We Need Redux?

Without Redux, managing state in large apps becomes hard:

- State is scattered across multiple components.
- Passing props deeply (prop-drilling) is messy.
- No centralized place to control app behavior.

Redux solves this by:

- Centralizing state in a single store.
 - Using a unidirectional data flow.
 - Making state transitions predictable.
-

◆ 3. How to Install Redux

Basic Redux and React-Redux

```
npm install redux react-redux
```

For DevTools Support

```
npm install redux-devtools-extension
```

◆ 4. Example: Counter and Chocolate Management App

Let's walk through the app you created:

✓ a. Initial State and Reducer

```
// reducer.js
const initialState = {
  counter: 0,
```

```

    totalchoc: 100,
    availchoc: 100
  };

export function reducercounter(state = initialState, action) {
  switch (action.type) {
    case "INCREMENT":
      return { ...state, counter: state.counter + 1 };

    case "buychoc":
      return { ...state, availchoc: state.availchoc - action.payload };

    case "returnchoc":
      return { ...state, availchoc: state.availchoc + action.payload };

    default:
      return state;
  }
}

```

Reducer Explanation:

- Handles different actions based on action.type.
- Uses spread operator (...state) to avoid mutating the original state.

b. Action Creators

```

// action.js
export function buychocAction(noofchoc) {
  return {
    type: "buychoc",
    payload: noofchoc
  };
}

export function returnchocAction(noofchoc) {
  return {
    type: "returnchoc",
    payload: noofchoc
  };
}

```

 These action creators return plain action objects used by the reducer.

c. Store Configuration

```
// store.js
import { legacy_createStore } from "redux";
import { reducercounter } from "./reducer";
import { composeWithDevTools } from "redux-devtools-extension";

export const reduxstore = legacy_createStore(reducercounter,
composeWithDevTools());
```

 composeWithDevTools() connects Redux to browser dev tools for easier debugging.

✓ d. React Component Using Redux

```
// Day20.js
import React from 'react';
import { useDispatch, useSelector } from 'react-redux';
import { buychocAction, returnchocAction } from './action';

function Day20() {
  const { availchoc, totalchoc, counter } = useSelector(state => state);
  const dispatch = useDispatch();

  const buychoc = () => {
    if (availchoc >= 3) dispatch(buychocAction(3));
    else alert("Not enough chocolates!");
  };

  const returnchoc = () => {
    if (availchoc + 3 <= totalchoc) dispatch(returnchocAction(3));
    else alert("Cannot return more than total!");
  };

  return (
    <div>
      <h2>Chocolate Counter</h2>
      <div>Total Chocolates: {totalchoc}</div>
      <div>Available Chocolates: {availchoc}</div>
      <div>Counter: {counter}</div>
      <button onClick={buychoc}>Buy 3 Chocolates</button>
      <button onClick={returnchoc}>Return 3 Chocolates</button>
    </div>
  );
}

export default Day20;
```

wrap up the component

```
<Provider store={reduxstore}>
  <Day20/>
```

```
</Provider>
```

◆ 5. Data Flow in Redux (Summary Diagram)

React Component

↓ dispatch(action)

Action (type, payload)

↓

Reducer

↓

New State

↓

React Component re-renders

◆ 6. Best Practices

- Always keep reducers pure.
 - Avoid direct state mutations (state.availchoc-- ❌).
 - Use constants for action types to avoid typos.
 - Consider Redux Toolkit for easier setup.
-

◆ 7. Redux DevTools

If installed, you can track every state change with time travel debugging.

🔗 Chrome Extension: [Redux DevTools](#)

How to fetch data from an API and manage it through Redux in a React application

📖 Complete Notes: Fetching Data and Managing It with Redux

🔄 Overview Workflow

1. **Fetch data** using axios in a React component.
 2. **Dispatch** an action with the fetched data.
 3. **Reducer** updates the Redux store with the data.
 4. **Component** reads and renders the data from the Redux store using useSelector.
-

1. Setup Reducer to Store Products

```
const initialstate = {
  products: []
};

export function productReducer(state = initialstate, action) {
  switch (action.type) {
    case "PRODUCT_SUCCESS":
      return { ...state, products: action.payload };

    case "add_product":
      return { ...state, products: [...state.products, action.payload] };

    case "delete_product":
      const id = action.payload;
      const newproducts = state.products.filter((item) => item.id !== id);
      return { ...state, products: newproducts };

    default:
      return state;
  }
}
```

- **Initial state:** An empty array of products.
 - **PRODUCT_SUCCESS:** Updates the state with API data.
 - **add_product:** Adds a new item to the list.
 - **delete_product:** Removes a product by ID.
-

2. Define Action Creators

```
export function productsAction(data) {
  return {
    type: "PRODUCT_SUCCESS",
    payload: data
  };
}

export function addproductAction(data) {
```

```

return {
  type: "add_product",
  payload: data
};
}

export function deleteproductAction(id) {
  return {
    type: "delete_product",
    payload: id
  };
}

```

Each function:

- Returns a plain object with type and payload.
- Is dispatched from components to modify the store.

3. Combine Reducers

```

import { combineReducers } from "redux";
import { productReducer } from "../products/reducer";

export const rootReducer = combineReducers({
  products: productReducer
});

```

Use combineReducers to manage multiple slices of state.

4. Create Redux Store

```

import { legacy_createStore } from "redux";
import { composeWithDevTools } from "redux-devtools-extension";
import { rootReducer } from "../combinereducer";

export const reduxstore = legacy_createStore(rootReducer,
  composeWithDevTools());

```

- Creates a store using reducers and connects it to Redux DevTools.

5. Provide Store to App

```

import { Provider } from "react-redux";

```

```
import { reduxstore } from './reduxstore';
```

```
<Provider store={reduxstore}>
```

```
  <App />
```

```
</Provider>
```

- Wrap your App component in <Provider> and pass the store to it.

6. Fetch and Display Products in Component

```
import React, { useEffect } from 'react';
```

```
import { useDispatch, useSelector } from 'react-redux';
```

```
import axios from 'axios';
```

```
import { productsAction, addproductAction, deleteproductAction } from './products/action';
```

```
function ProductComponent() {
```

```
  const { products } = useSelector(state => state.products);
```

```
  const dispatch = useDispatch();
```

```
  useEffect(() => {
```

```
    fetchdata();
```

```
  }, []);
```

```
  async function fetchdata() {
```

```
    const res = await axios.get("https://dummyjson.com/products");
```

```
    dispatch(productsAction(res.data.products));
```

```
  }
```

```
  const addProduct = () => {
```

```
    const newProduct = {
```

```
      id: 31,
```

```
      title: "Samsung S23",
```

```
      description: "Flagship phone",
```

```
      category: "electronics",
```

```
      price: 999
```

```
    };
```

```
    dispatch(addproductAction(newProduct));
```

```
  };
```

```
  const deleteProduct = (item) => {
```

```
    dispatch(deleteproductAction(item.id));
```

```
  };
```

```
  return (
```

```
    <div>
```

```
      <button onClick={addProduct}>Add Samsung S23</button>
```

```
      {products.map((item) => (
```

```
        <div key={item.id}>
```

```
    <h3>{item.title}</h3>
    <button onClick={() => deleteProduct(item)}>Delete</button>
  </div>
  )))
</div>
);
}
```

Redux Flow Summary

Step	Action
1	Component mounts, useEffect triggers fetchData()
2	API response is dispatched via productsAction()
3	Reducer stores the data in Redux under products
4	useSelector reads state and maps over products to display
5	addproductAction() & deleteproductAction() modify list

Benefits of Using Redux for Data Management

- **Global state:** Accessible from any component.
 - **Predictability:** State changes follow consistent rules.
 - **Debugging:** Redux DevTools provide history and time travel.
 - **Centralized logic:** Actions and reducers keep logic organized.
-